

“华为杯”第二十二届中国研究生 数学建模竞赛

题 目： 低空湍流监测及最优航路规划研究

摘 要：

低空湍流是空气流动中因速度、压力或方向剧烈变化而产生的无序、不规则运动状态，因近年低空经济发展空前高涨，对低空湍流的特征分析需求大幅上升。本文基于湍流强度计算的数学方法，建立了**基于全时空信息的湍流强度最优计算模型**、**基于风廓线雷达数据的湍流计算模型**等五个模型，对低空区域范围内的湍流强度分布特征进行精细刻画，对飞行器**最优航路**的监测与规划进行研究。

针对问题一，首先综合考虑风廓线雷达数据与微波辐射计的数据，运用**双重插值**将二维的时间、空间数据匹配合并，以此作为全时空信息的湍流强度最优计算模型原始数据，计算湍流强度即湍流动能在**低空范围内**的时空分布，给**单资料二维湍流计算模型**构建基准数据集。接着，通过风廓线数据的雷达**反演研究**并构建单资料二维湍流计算模型。此模型基于机器学习，采用分层 **Bootstrap** 抽样使用 70%的原始数据作为训练集构建**随机森林模型**，用**原始数据 30%**测试集和最优计算模型结果进行双重验证得到拟合优度为 0.497，基于 **XGBoost** 优化模型的泛化能力，将拟合优度提升至 0.562。

针对问题二，研究基于同化融合后的多源异构资料**三维空间**湍流强度计算模型。首先，对地面气象站、风廓线雷达、多普勒雷达数据进行**特征以及相关性的分析**，互相作为数据的补充条件。接着，通过**数据清洗**，剔除异常值，本文基于 **VAD** 技术原理，通过以下三步将多普勒雷达的径向速度数据**重建水平风场**：（1）在水平方向上分解风向量（2）建立观测方程（3）通过**最小二乘**求解重建的风场。然后采用**加权融合**的方法，赋予 X 波段 0.6 的权重，S 波段 0.4 的权重，构建高质量背景场，将坐标都转换为**笛卡尔坐标系**，利用简化的**集合卡尔曼滤波插值法**，得到三维融合风场，结合**湍流动能**以及**耗散率**从两个角度分析气象站覆盖区域范围内的湍流强度空间分布。最后考虑**地理信息因素**，考虑地形以及城市区域对低空湍流强度的分布特征的影响。

针对问题三，研究低空飞行器的湍流预警以及最优航路规划。本文首先构建了基于**数值天气预报数据**的湍流强度场，通过天气预报气象场数据，从中挑选出需要的因素，并根据不同的时空分辨率要求，生成了一个基于**湍流动能**为主要指标的新 TKE 场，并依据这能量场进行最优航路规划。在最优航路规划中，本文以**代价函数**作为最优航路的规划指标，通过给定起点和终点，结果最优路径的代价少于其它模型，也证明了最优路径的合理性。本文还利用**多种探测设备**，基于机器学习中的 **LSTM** 模型分别对不同设备，通过**径向基插值**和 **k-means** 聚类预测出需要的预测湍流场，其次通过**拟合优度归一化**对不同设备预测结

果加权平均，得到**模型加权**后的湍流预测场，在该预测场的最优航路规划问题上，采用**三维蚁群算法**，通过从起点释放一定数量的“蚂蚁”，从中选出最优路径，即为飞行器所参考的最优路径。

本文聚焦于**低空飞行器发展瓶颈**，考虑低空飞行的不稳定性，构建了五个既相关又递进的模型，从双资料计算湍流强度，到单设备资料模型的构建，再结合**多源异构数据分析**模型，从不同维度验证湍流强度空间分布的特征。最终利用湍流强度的空间分布特征，为低空飞行器规划最优路径，保证飞行器的安全，促进**低空经济**的发展。

关键词：XGBoost; 湍流动能; LSTM 模型; 代价函数;

目 录

一、问题重述	4
二、问题分析	4
三、模型假设	4
四、符号说明	5
五、问题一：基于单资料二维湍流计算模型的研究.....	5
5.1 基于全时空信息的湍流强度最优计算模型.....	5
5.1.1 全时空湍流强度计算模型的构建.....	5
5.1.2 计算湍流动能并基于理查逊数分析湍流强度	7
5.1.3 低空区域湍流强度时空分布的研究.....	8
5.2 基于风廓线雷达数据的湍流计算模型研究.....	10
5.2.1 基于随机森林的湍流计算模型原理.....	10
5.2.2 基于分层抽样优化随机森林模型.....	11
5.3 基于最优模型校准精确研究.....	11
5.3.1 基于基准真值对 b 模型的性能评估.....	12
5.3.2 基于 XGBoost 优化模型 B 原理.....	13
5.3.3 随机森林模型与 XGBoost 模型的残差对比.....	14
六、问题二：基于同化资料的三维空间湍流强度计算模型.....	15
6.1 基于异构数据的三维湍流强度模型构建.....	15
6.1.1 多源异构数据的特征分析.....	15
6.1.2 多源异构数据的同化与融合	16
6.2 涡旋耗散率计算湍流强度.....	18
6.3 基于多角度分析三维空间湍流强度计算模型的准确性.....	18
6.3.1 三维湍流强度、三维湍流动能空间分布研究.....	19
6.3.2 基于地理信息空间湍流强度研究.....	20
七、问题三：基于多尺度数据的低空湍流场及航路规划策略研究.....	23
7.1 基于预报天气的湍流强度场最优航路规划.....	23
7.1.1 基于数值天气预报数据湍流强度场的构建.....	23
7.1.2 基于标准尺度新型湍流强度场的最优航路预测	27
7.2 基于多种观测资料的湍流强度场最优航路规划.....	32
7.2.1 基于 LSTM 对风廓线雷达设备数据的湍流动能场预测模型构建	32
7.2.2 基于 LSTM 对 S 波段天气雷达设备数据的湍流动能场预测模型构建	34
7.2.3 基于模型加权的湍流动能场最优航路规划.....	36
参考文献	39
附录	41

一、问题重述

近年来，低空经济作为国家战略性新兴产业迅速崛起，代表着我国的低空经济发展热度持续升温，然而低空经济形态是以高度 1000 米以下的空域为活动空间为基准，面临的环境复杂程度越来越大，覆盖城市、山地、水域和森林等多种地形地貌，给一般的低空飞行器的使用造成极大的困扰。飞行器易受到地形扰动、热力对流和局地风场变化等多重影响因素影响，而大气中因速度、温度或者方向剧烈变化所引发的不规则流动现象又被称为湍流。为了保障低空飞行器飞行的安全，结合低空区域及该区域内的多种探测设备资料，探究湍流强度三维空间的分布特征，进行飞行湍流的监测及预警，从空间和时间两个维度上开展研究，据此研究如何有效规划飞行航路。

二、问题分析

针对问题一，综合使用风廓线雷达和微波辐射计两种设备的资料，运用多样化数据，建立模型 a 用于计算不同高度上湍流强度指标，同时，仅利用风廓线雷达资料，建立计算湍流强度指标的模型 b，以模型 a 的结果作为评判标准，对模型 b 进行验证与优化，最终确立一个仅基于风廓线雷达资料的高精度、高效率、可靠的湍流强度指标计算模型。

针对问题二，同化融合地面自动气象站、风廓线雷达、多普勒天气雷达等多种设备的探测资料，融合异构数据，构建一个覆盖指定区域的三维空间湍流强度计算模型。该模型需满足低空飞行所需的时空分辨率，并最终结合实际地理信息，多角度可视化显示湍流强度的三维分布结果。

针对问题三，利用连续三小时的观测资料计算得到的湍流强度作为验证标准，研发一个基于数值天气预报数据的湍流强度预报模型 d，并对模型结果进行分析与优化。进而利用该预报模型，对未来时段的三维湍流进行预报，并在给定约束条件下，规划出一条仅考虑湍流条件的最优飞行航路。此外，还需要构建一个仅基于短期观测数据的非线性外推模型 e，实现对未来一小时的湍流预测与航路规划，并分析其可靠性。

三、模型假设

为了使建立的模型得到简化，做出以下假设：

- 假设各探测设备在分析时段内的观测数据在时间和空间上可对齐，其数据质量满足湍流分析的基本要求；
- 假设在计算特定湍流参数时，可应用基于局部各向同性湍流理论的经典关系式；
- 假设大气变量场在时空演变上具有连续性，为数据插值与短期外推提供了依据；
- 航路优化问题在当前阶段简化为仅以规避湍流和为核心目标。

四、符号说明

符号	含义
Ri	理查逊数，表征大气稳定度
TKE	湍流动能，反映湍流运动剧烈程度
ε	涡旋耗散率，表征湍流能量耗散速率
g	重力加速度
θ	位温，消除气压影响的热力参数
(u, v, w)	风速在东西、南北、垂直方向的分量

五、问题一：基于单资料二维湍流计算模型的研究

当前湍流监测设备技术正处于从多设备协同向单设备检测的过渡期，因此以本文构建的综合考虑空间位置近似重合的风廓线雷达、微波辐射数据的模型为基准，对比验证仅考虑风廓线雷达单一数据的模型的可靠性，以风廓线雷达资料湍流强度最优计算模型为目标，推动智能单设备检测湍流强度的发展进程。

5.1 基于全时空信息的湍流强度最优计算模型

该模型为模型 a，将风廓线雷达数据以及微波辐射计数据同化融合，以合并后的数据作为输入，以湍流强度相关物理原理作为数学模型的核心要义，辅佐理查逊数数学算法，计算湍流强度，最终得出湍流强度在低空区域内的时空分布特征，如图 1 所示。

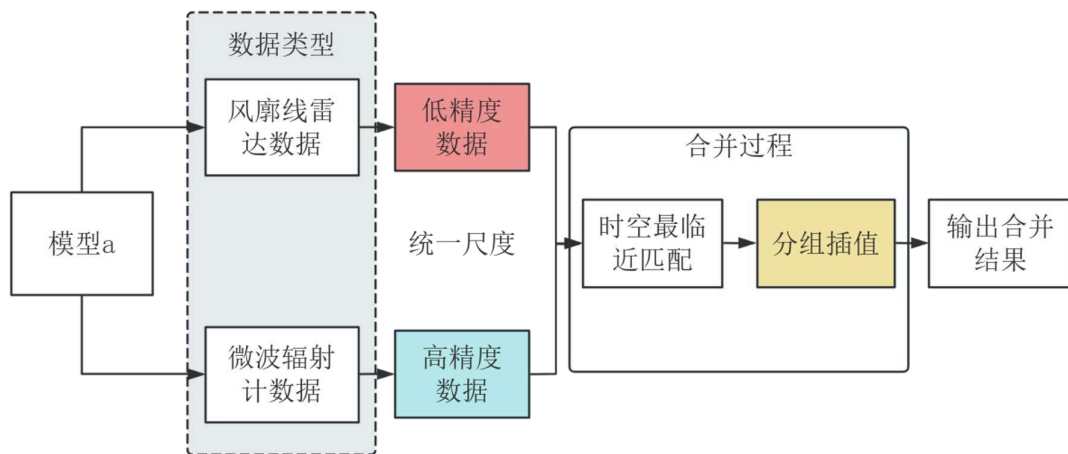


图 1 基于双资料二维湍流计算模型的基准模型结构图

5.1.1 全时空湍流强度计算模型的构建

(1) 风廓线雷达数据与微波辐射计数据的同化融合

为了综合利用风廓线雷达及微波辐射计两种设备的资料，本文将风廓线雷达数据与微波辐射计数据进行时空匹配合并，首先对数据集按时间排序 D_w 、 D_r ，对 D_w 中每个时间

点 t_w ，在 D_r 中查找满足 $|t_w - t_r| \leq \Delta t$ 的最近 t_r ，接着计算高度差 $\Delta h = |h_w - h_r|$ ，通过设定过滤条件，使得 $\Delta h \leq \text{定值}$ ，将匹配后的数据按高度分组，运用双向插值法对每个高度层的时间序列进行插值，形成合并后的数据集，数据处理结构如图 2 所示。

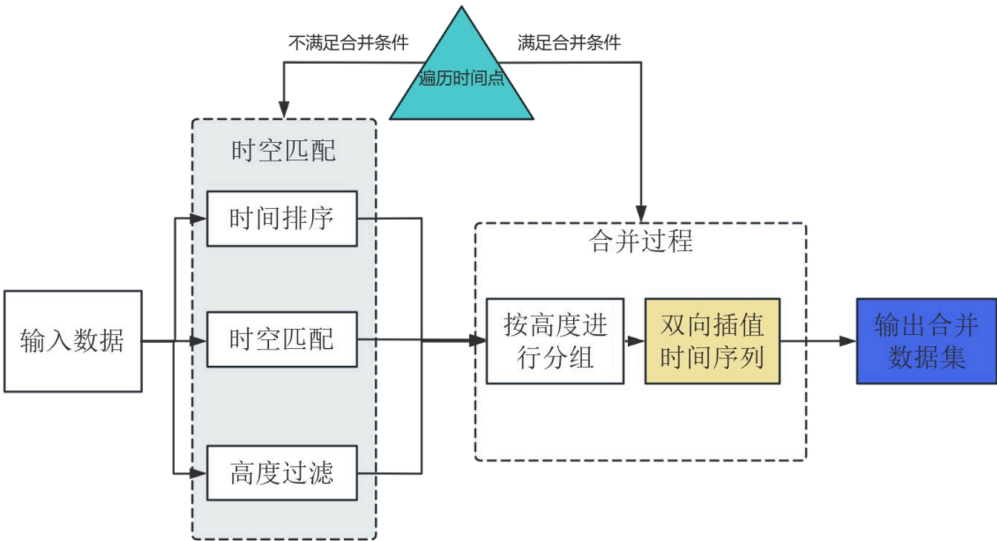


图 2 风廓线数据与微波辐射计数据融合算法流程图

(2) 风廓线 a 站点和风廓线 b 站点的数据对比

将 a 站点风廓线雷达数据和 b 站点风廓线雷达数据进行对比分析。

高度-风速对比

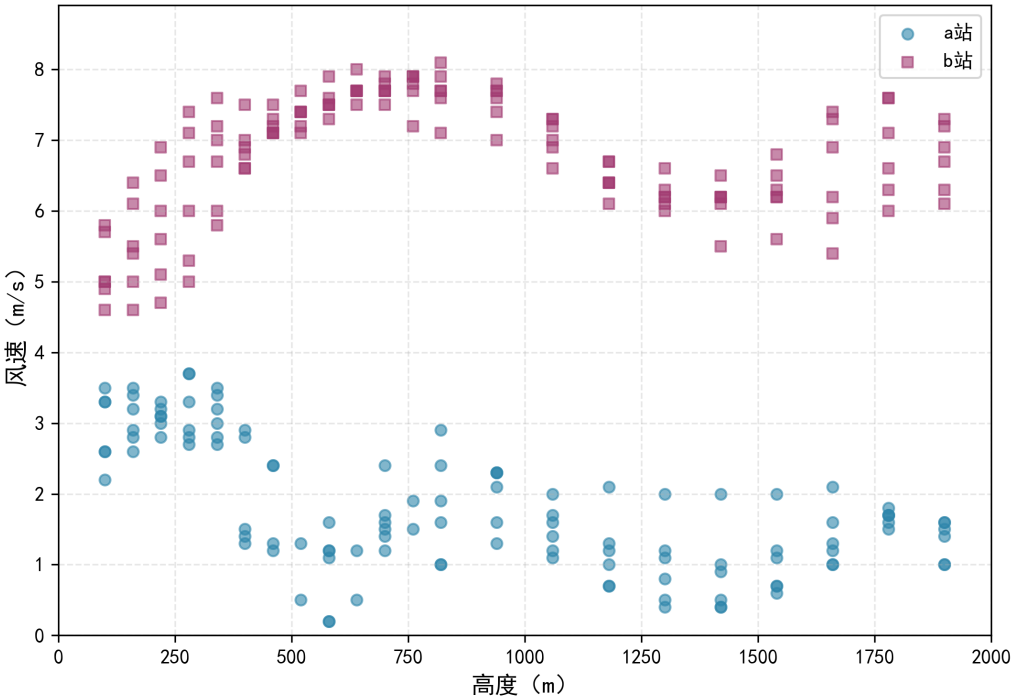


图 3 风廓线 a、b 站点高度与风速对比分析图

从图 3 中观察到 a 站与 b 站之间的数据存在显著差异。从风速整体水平上来看 a 站的

风速整体低于 b 站，本文发现高度与风速之间的关联，b 站在 750m 以下存在较高风速分布，且随着高度变化，风速波动中高值占比大，而 a 站风速随高度变化，整体处于相对较低的水平，高风速情况较少。

5.1.2 计算湍流动能并基于理查逊数分析湍流强度

(1) 位温计算

位温指气块在干绝热过程中被压缩或膨胀至标准气压时的温度，是描述气块热力状态的重要参数，意义在于消除气压对温度的影响，便于比较不同高度空气块的热力性质。变量 θ 为位温， T 为气温， P_0 为标准大气压 1000hPa， P 为实际大气压， R 为干空气的比气体常数， C_p 为干空气的定压比热容。

$$\theta = T \times \left(\frac{P_0}{P} \right)^{\frac{R}{C_p}}$$

其中 $\frac{R}{C_p}$ 是一个近似常数，约等于 0.286，因此上述公式可以简化为：

$$\theta = T \times \left(\frac{2000}{P} \right)^{0.286}$$

(2) 理查逊数计算

对于理查逊数计算，基于已匹配合并的风场数据，读取 u 、 v 、 z 相关变量的数据，通过数值微分求解垂直风切变，结合微波辐射计提供的温度数据计算大气垂直温度梯度，以 g 为重力加速度， θ_v 为上述计算的位温， dz 为垂直高度差， du/dz 、 dv/dz 分别为水平风场在垂直方向的切变。

代入理查逊数公式：

$$Ri = \frac{g}{\theta_v} \cdot \frac{d\theta_v}{dz} \cdot \frac{(du/dz)^2}{(du/dz)^2 + (dv/dz)^2}$$

计算得到不同时刻、不同高度层的 Ri ，展现湍流强度的空间分布。

(3) TKE 湍流动能的计算

通过 Ri 值判断湍流发展状态后，再对湍流动能进行计算，利用风场数据分解大气运动为平均运动与脉动运动，通过原始风场数据减去时间平均风场得到湍流脉动速度分量，最后结合温度湿度的数据对大气稳定度进行修正，代入 TKE 计算公式。

$$\begin{cases} TKE = \frac{1}{2}(u'^2 + v'^2 + w'^2) \\ u' = u - \bar{u} \\ v' = v - \bar{v} \\ w' = w - \bar{w} \end{cases}$$

这里 u 、 v 、 w 分别代表实际风在 x 、 y 、 z 方向上的分量， $\bar{u}$ 、 $\bar{v}$ 、 $\bar{w}$ 分别代表 u 、 v 、 w 的平均值， u' 、 v' 、 w' 分别为风速分量在 x 、 y 、 z 方向上的脉动值。以此直接量化湍流能量

大小，最终综合两类算法结果，形成可表征大气湍流强度的量化指标，作为湍流强度评估的标准输出，以此可以得到 a 模型在 a 站点和 b 站点低空区域的湍流强度，综述计算算法流程如图 4。

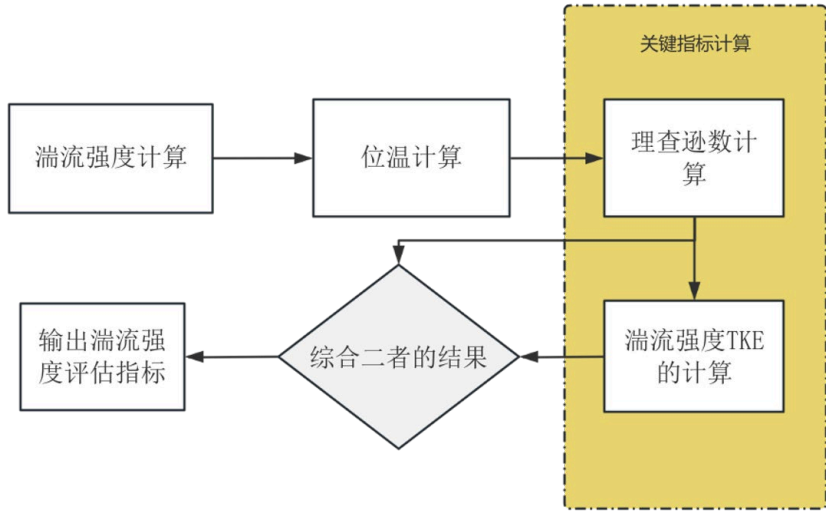


图 4 湍流强度计算流程图

5.1.3 低空区域湍流强度时空分布的研究

(1) 模型 a 在风廓线雷达 a 站点湍流动能与 Ri 的一致性

通过六个时间点，湍流动能 TKE 大小随高度分布的垂直剖面，由图 5 可知，多个高度层存在湍流的波动，在 1000m 以下区域湍流动能的曲线波动较为剧烈，说明低空大气层结构不稳定，易形成对流等影响低空飞行器的安全，在高空区域则波动较少，大气层稳定。

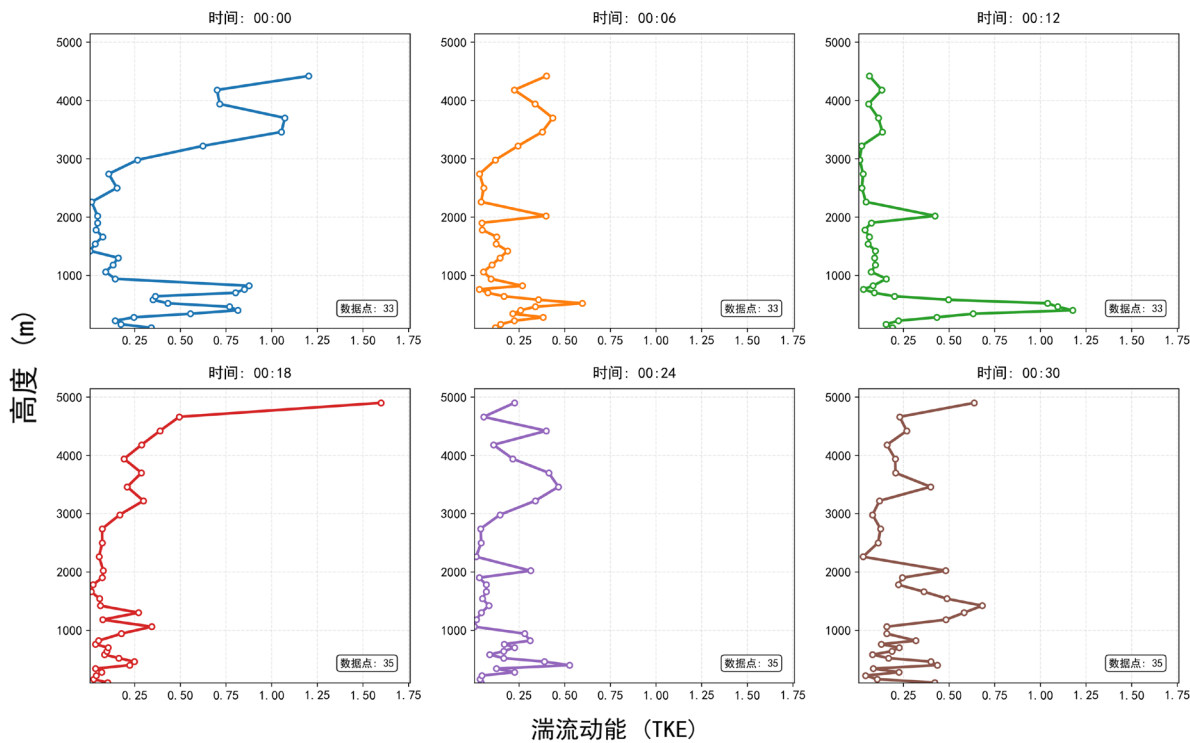


图 5 模型 a 在 a 站点湍流动能 TKE 不同时刻垂直剖面图

Ri 数值较大时，代表湍流强度较弱，数值越小湍流越剧烈，结合 Ri 在六个时间点，随高度变化的垂直剖面图，可对模型 a 的计算结果进行验证，由图 6 可知， Ri 在 1000m 以下数值较小，且波动密集，与模型 a 湍流动能 TKE 的计算结果吻合，在高空区域 Ri 数值较大，且波动曲折线稀疏，代表大气层稳定。

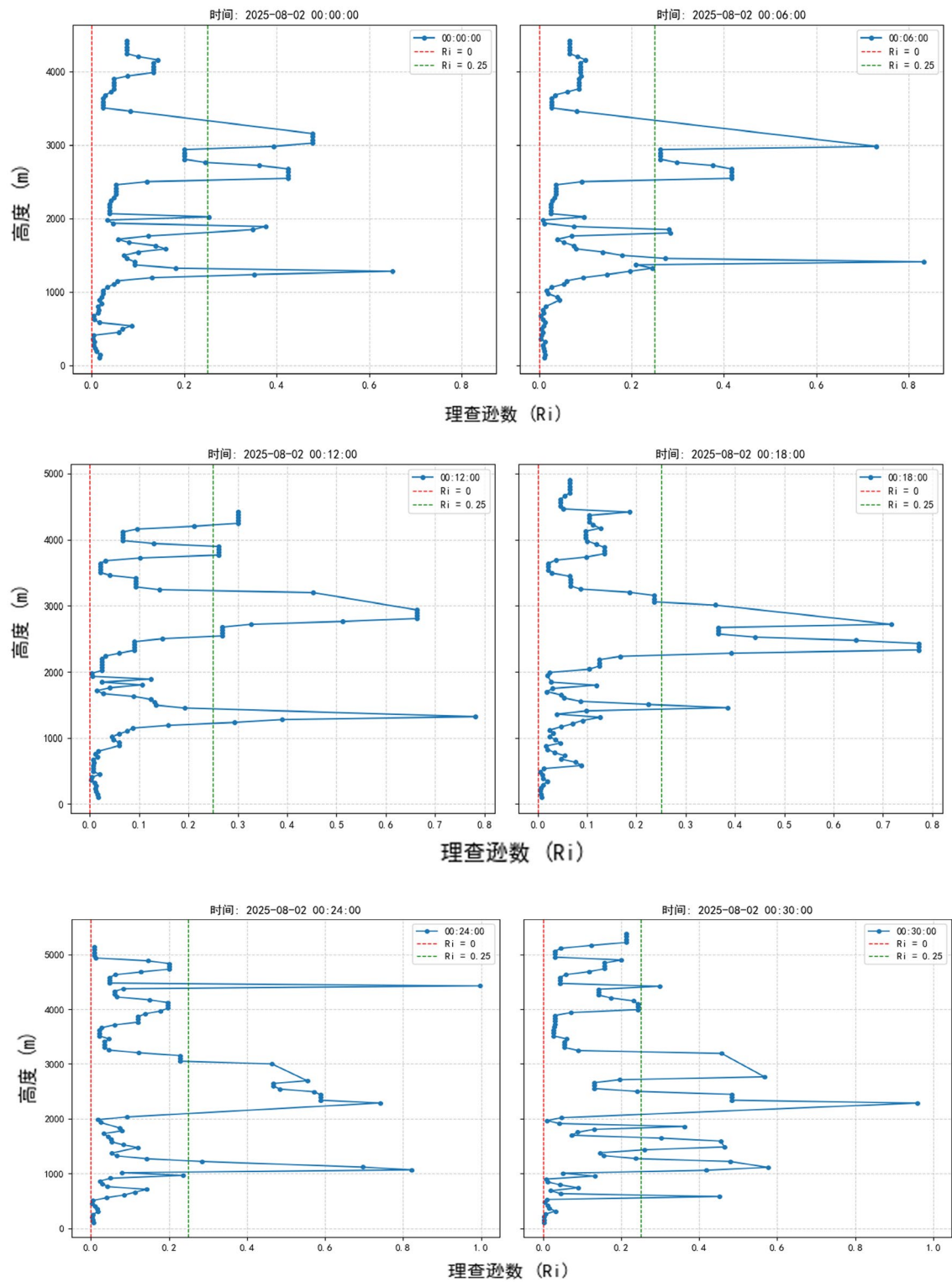


图 6 Ri 在不同时刻垂直剖面图

(2) 模型 a 在风廓线雷达 a、b 站点 TKE 热力分析

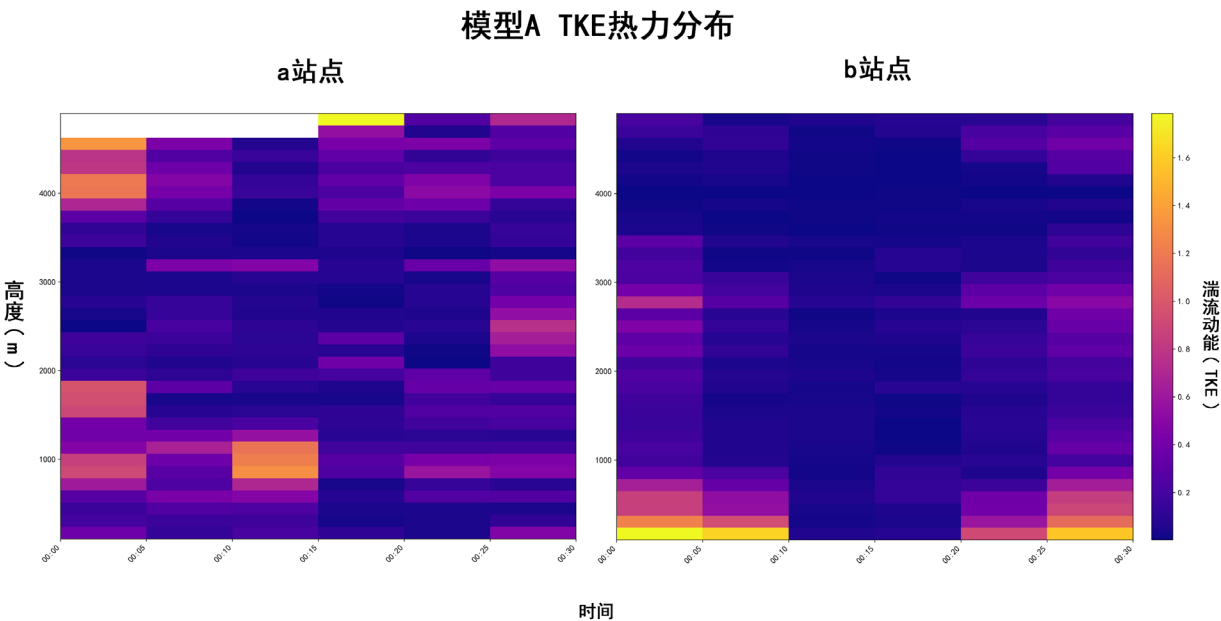


图 7 模型 a 在不同风廓线雷达站的 TKE 热力分布

由图可以体现，近地面湍流动能较大，已出现薄弱湍流，大约在 2000m 湍流动能较小，大气层结构稳定，与前文结论具有一致性。

5.2 基于风廓线雷达数据的湍流计算模型研究

该模型为模型 b，仅基于风廓线雷达的湍流强度计算同样可以利用风场数据分解大气运动为平均运动与脉动运动，基于通过原始风场数据减去时间平均风场得到湍流脉动速度分量，结合温度、湿度数据对大气稳定度的修正，代入 TKE 计算公式，如图 8 所示。

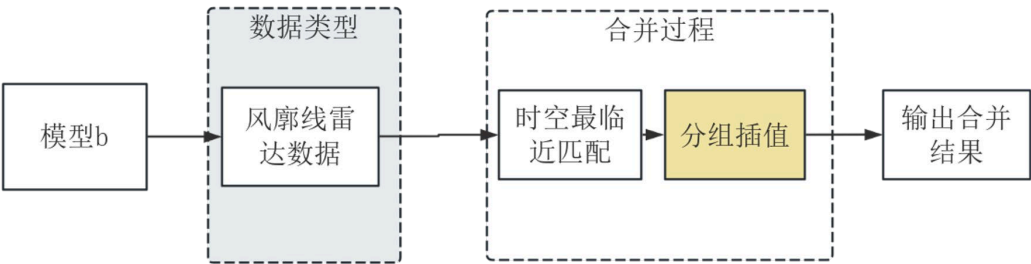


图 8 基于风廓线雷达数据的湍流计算模型结构图

5.2.1 基于随机森林的湍流计算模型原理

该模型采用了机器学习的方法，仅读取风廓线雷达的数据，运用风速数据计算 TKE 湍流动能。目的为构建风廓线数据与湍流强度之间的映射关系，本研究以 a、b 站点的风廓线雷达观测值处理后的合并数据集为基础数据集，采用**随机森林**回归算法建立预测模型，其算法循环流程如图 9。

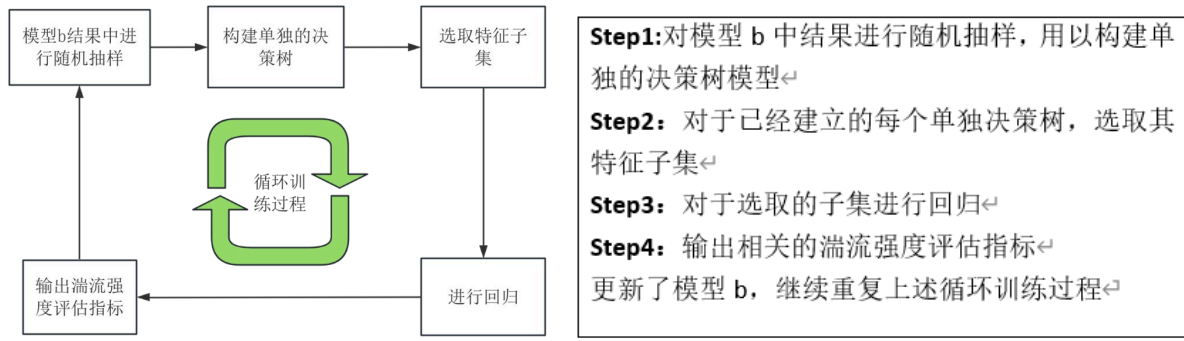


图9 随机森林回归算法建立预测模型结构及分层 STEP 步骤

随机森林的默认算法步骤如下，首先，对原数据集进行 **Bootstrap 抽样**，即从训练集中有放回地随机抽取多个样本子集，然后并行建树，用每个样本子集独立训练一棵决策树，在树的分裂节点时，仅从随机选取的特征子集中寻找最佳分裂点，对于新给出的样本，每棵树给出一个预测值，最终的预测结果是所有树预测值的平均值，就是运用了回归的思想，最终预测值可以表示为：

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

其中， B 是树的总数， $T_b(x)$ 是第 b 棵树的预测值。

回归准则通常是使用均方误差最小化来度量的，对于一个节点，其 MSE 为：

$$\text{MSE}_{\text{node}} = \frac{1}{n_{\text{node}}} \sum_{i \in \text{node}} (y_i - \bar{y}_{\text{node}})^2$$

选择能最大程度降低 MSE 的特征和阈值进行分裂。在这个问题中，该机器学习算法可用于学习从雷达数据到湍流强度的复杂映射关系，从而构建预测模型。该模型以位置信息与高度作为输入自变量，以 TKE 值为输出因变量。

5.2.2 基于分层抽样优化随机森林模型

由于存在站点 a 和站点 b 的位置分类变量，为避免因站点样本分布不均衡导致模型对特定站点预测存在系统性偏差，采用以站点号为分层变量的分层抽样策略，分别从站点 a 和站点 b 中各自抽取 70% 数据作为总的训练集，通过该策略能够避免出现普通 Bootstrap 抽样结果出现 a 站点数据信息抽取量明显大于 b 站点的情况，体现了该模型对空间变化反映的可靠性。

在随机森林模型训练阶段，为进一步提升训练样本的代表性，降低单一随机抽样的不确定性，在算法原生 bootstrap 有放回抽样前，先对目标变量 TKE 值进行区间分层，再在各目标值区间内执行有放回抽样，以此保证每棵决策树的训练样本均涵盖不同 TKE 值区间的特征，最终设定随机森林树的数量为 100 棵以实现集成学习效应；模型训练完成后，将测试集的位置与高度信息输入该 b 模型得到 TKE 预测值。

5.3 基于最优模型校准精确研究

本文以全时空信息的湍流强度最优计算模型即模型 a 为基准，对基于风廓线雷达数据的湍流计算模型的结果从均方误差、决定系数两方面进行分析，判断模型 b 的正确性，从而采用 XGBoost 模型进行优化。

5.3.1 基于基准真值对 b 模型的性能评估

(1) 模型预测值与基准真值间的差异反映

对随机森林回归构建的 b 模型进行性能评估，首先对模型 a 用构建模型 b 时选取的测试集输出一个 TKE 值为，将该值记为基准真值，其次用训练好的模型 b 在同一测试集上计算出预测结果，计算基准真值与训练模型预测的核心评估指标如模型的均方误差（MSE）为 0.0425，能够反映模型预测值与基准真值间平方误差的平均值，其数值较小，表明整体误差水平较低；决定系数（R²）为 0.4977，表明该模型能够解释基准真值约 49.77%的变异程度，模型具备一定的拟合与预测能力。随机森林结果残差分布图 10 基本符合正态分布且以 0 为中心对称的特征，说明模型在残差的统计特性上表现较好，大概率满足线性模型的关键假设，该湍流强度模型的预测误差具有随机性与对称性，无明显的系统偏差，整体拟合效果较为理想。

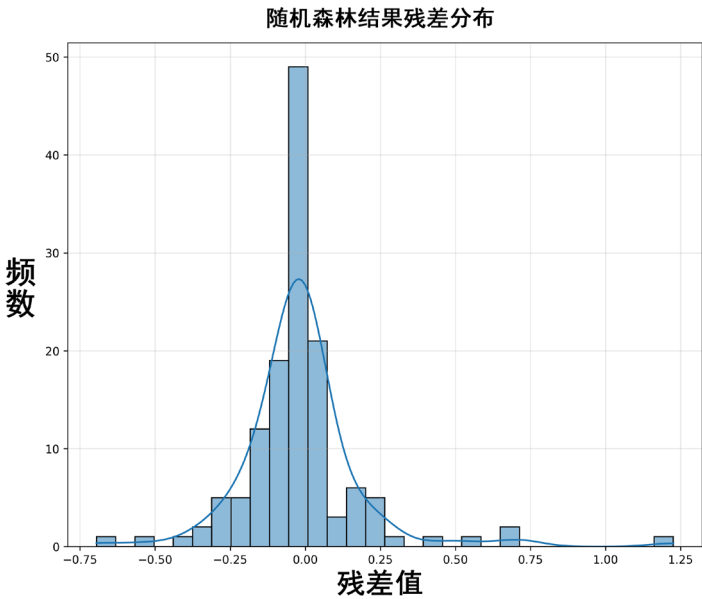


图 10 随机森林残差分布图

这种评估方法即利用了模型 a 来验证模型 b，又考虑到了机器学习过程中本身就含有对训练模型的评估，综合利用了二者的验证手段，对模型 b 进行双重验证，以保证验证过程的严谨性。

(2) 模型在空间位置上的差异反映

由抽样过程中采用的分层抽样，因此我们可以对分层部分即对站点 a 和站点 b 各自位置的数据进行误差分析。

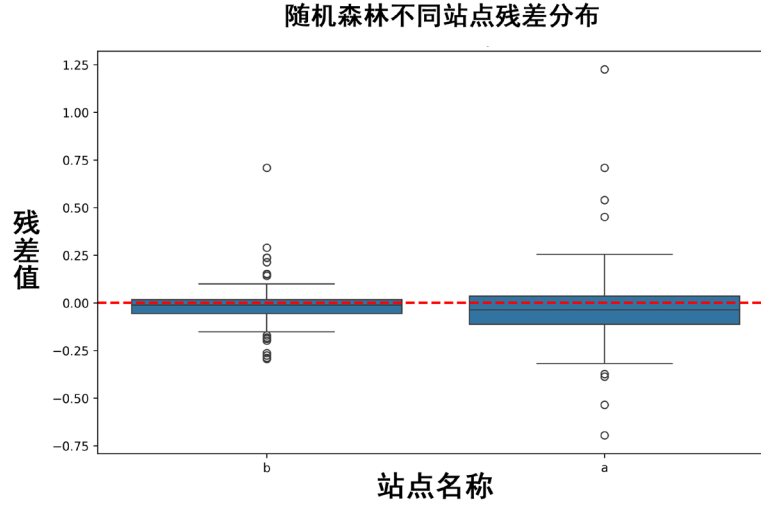


图 11 随机森林不同站点残差分布箱线图

从残差分布图 11 来看，b 站的模型残差更接近 0、波动更小、离群点更少，模型预测的准确性与稳定性优于 a 站；而 a 站残差波动大、离群点多，预测偏差的一致性与稳定性相对较弱。该模型在相同时间与高度条件下，不同站点 TKE 预测值存在显著差异，反映出模型能够有效捕捉站点位置这一关键自变量对 TKE 值的影响

5.3.2 基于 XGBoost 优化模型 B 原理

在相同参数与训练条件下，XGBoost 相较于随机森林的核心优化，源于从 **Bagging** 并行集成框架向 **Boosting** 串行集成框架的底层逻辑跃迁，并通过算法设计与工程实现的多维升级形成性能优势。

XGBoost 通过迭代训练 K 个弱学习器，最终预测输出为各树预测值之和，即：

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i)$$

其中， $\hat{y}_i$ 为样本 i 的最终预测值， $f_k(x_i)$ 为第 k 棵树对样本 i 的预测值， x_i ：第 i 个输入样本。

我们的目的，则是使得目标函数最小，

$$\mathcal{L}^{(k)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(k-1)} + f_k(x_i)) + \Omega(f_k)$$

其中， $\mathcal{L}^{(k)}$ 为第 k 轮迭代的目标函数， $l(\cdot)$ 为损失函数； $\hat{y}_i^{(k-1)}$ 为前 $k-1$ 棵树对样本 i 的累计预测值； $\Omega(f_k)$ 为第 k 棵树的正则项，形式为：

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

其中 T 为树的叶子节点数， w_j 为第 j 个叶子节点的输出权重； γ 、 λ 为正则化系数，分别控制树复杂度与权重大小。

基于以上原理，本文使用 XGBoost 模型进行拟合。

表 1 随机森林与 XGBoost 特点对比

对比	随机森林	XGBoost
训练方式	并行独立训练	串行依赖训练
样本利用方式	Bootstrap 抽样	动态调整样本权重
集成目标	降低方差	优先降低偏差，然后再控制方差

5.3.3 随机森林模型与 XGBoost 模型的残差对比

XGBoost 的残差分布图也基本符合正态分布且以 0 为中心对称的特征，整体拟合效果较为理想。

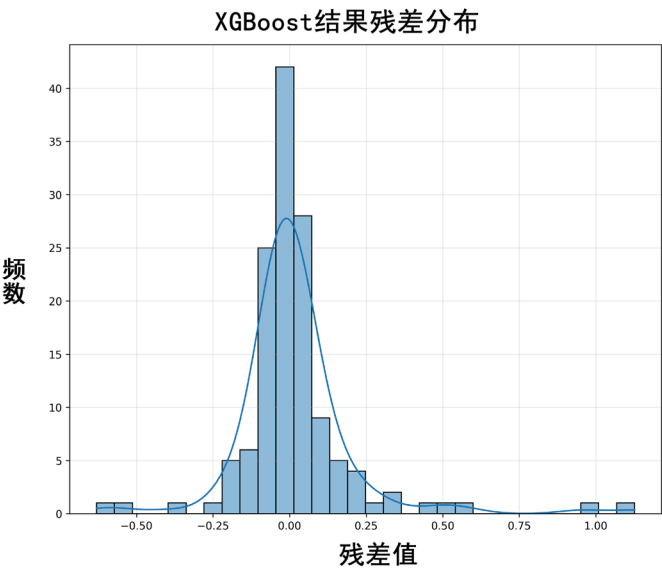


图 12 XGBoost 残差分布图

同样由下列残差图可知，使用该模型的效果结论与随机森林类似，核心结论不变。

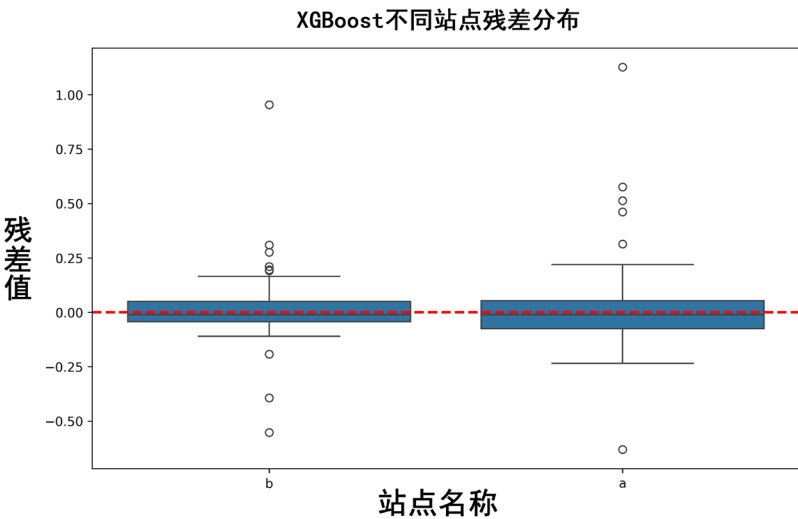


图 13 XGBoost 不同站点残差分布箱线图

将模型从随机森林优化为 XGBoost 后，预测性能实现全面且显著的提升：从误差指标来看，XGBoost 的均方误差 $MSE=0.0370$ 、均方根误差 $RMSE=0.1924$ 、平均绝对误差 $MAE=0.1045$ ，分别较随机森林 $MSE=0.0425$ 、 $RMSE=0.2061$ 、 $MAE=0.1184$ 下降约 13%、6.7%、11.7%，对预测值与真实值的偏差控制更优，尤其在降低对异常值敏感的平方误差上效果突出，预测精度显著提高；从拟合优度来看，XGBoost 的决定系数 $R^2=0.5621$ 较随机森林 $R^2=0.4977$ 提升约 13%，能多解释数据集中 6.44% 的因变量变异，对数据规律的捕捉能力更强，整体拟合效果更优，综上，此次优化有效。

六、问题二：基于同化资料的三维空间湍流强度计算模型

由于低空飞行场景复杂，飞行区域包括城市、森林、农田等，极大程度受到了地形导致的局地气流作用的影响。因此，问题二综合考虑了多种设备提供的探测资料，弥补单一设备的局限性，需要融合不同来源的数据，构建一个三维空间上的湍流强度计算模型。

6.1 基于异构数据的三维湍流强度模型构建

该模型为模型 c，首先通过资料同化技术，将地面自动气象站、风廓线雷达和多普勒天气雷达的观测数据处理成三维风场，接着计算相关湍流强度指标，通过耗散率和湍流动能多角度显示湍流的强度，最终输出覆盖所有地面自动气象站的矩形区域、水平分辨率达到 100m，垂直分辨率达到 50m 并且包含地理信息数据的三维结果。

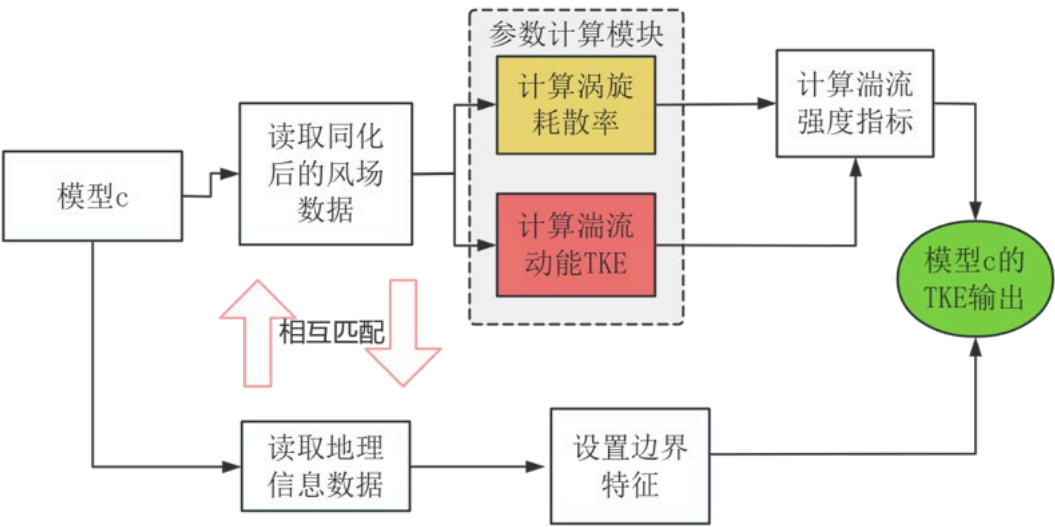


图 14 基于异构数据的三维湍流强度模型结构示意图

6.1.1 多源异构数据的特征分析

地面自动气象站、风廓线雷达、多普勒雷达的数据具有不同的要素以及数据结构，其中地面自动气象的优势在于拥有近地面数据如：风速、气温、湿度等，分辨率较高但缺少垂直剖面的信息，站点稀疏覆盖率低。风廓线雷达垂直结构完备刚好补充了地面气象站数据的劣势，a、b、c 三个站点呈现三角状结构，覆盖率更广，单只包含风场数据，且易受

外部条件干扰。多普勒雷达数据能够探测到云和降水区的风场覆盖范围大，分辨率高，但在晴空探测条件有限。

6.1.2 多源异构数据的同化与融合

地面自动气象站、风廓线雷达、多普勒天气雷达数据探测**角度多元化**，包含要素各不相同，时空分辨率也存在差异，因此需要将多源异构数据进行同化，统一使用笛卡尔坐标系，综合考虑多种探测设备的优缺点，“取长补短”互相的要素内容作为补充条件，使得通话后的数据更准确、全面，为模型的实施创造条件。

(1) 重新构建水平风场

首先进行**数据清洗**，将多普勒雷达中-9999的数值，替换成NaN，利用**VAD技术**，将多普勒雷达的径向速度数据**重建水平风场**，可分成三步：

- 1、将水平风场分解为 u 和 v 分量
- 2、建立 $V_r = u \sin \theta + v \cos \theta$ 的观测方程
- 3、通过最小二乘求解超定方程组

多普勒雷达测量的径向速度 (V_r) 与真实风场 ($\vec{V} = (u, v, w)$) 满足：

$$V_r = \vec{V} \cdot \hat{r} = u \cos \theta \sin \phi + v \sin \theta \sin \phi + w \cos \phi$$

其中： θ 为方位角； ϕ 为仰角。

只考虑水平风场，即仰角度数为 0，可以将公式简化为 $V_r = u \cos \theta + v \sin \theta$ 。

对 N 个方位角观测值 (θ_i, V_{r_i}) ，构建线性方程组，利用最小二乘法重新求出水平风场：

对于方程组：

$$\begin{cases} V_{r_1} = u \cos \theta_1 + v \sin \theta_1 \\ V_{r_2} = u \cos \theta_2 + v \sin \theta_2 \\ \vdots \\ V_{r_N} = u \cos \theta_N + v \sin \theta_N \end{cases}$$

用矩阵形式来表达：

$$V_r = Ax + \varepsilon$$

$$A = \begin{bmatrix} \cos \theta_1 & \sin \theta_1 \\ \cos \theta_2 & \sin \theta_2 \\ \vdots & \vdots \\ \cos \theta_n & \sin \theta_n \end{bmatrix}, \quad x = \begin{bmatrix} u \\ v \end{bmatrix}, \quad V_r = \begin{bmatrix} V_r \\ V_c \end{bmatrix}$$

最小二乘解为：

$$\hat{x} = (A^T A)^{-1} A^T V_r$$

接着以雷达地理位置为中心，以设备能够探测到的最大距离作为边界半径，确保包含所有有效的数据点并且能够覆盖所有有效探测范围，创建 100×100 的规则均匀网格区域，该网格区域是在**笛卡尔坐标系**下均匀分布的正交网格，然后将由**VAD分析**得到的 u 、 v 风

向量利用矩阵插入，从而构建多普勒雷达的水平均匀风场。

(2) 加权融合构建背景场

通过对 X 波段多普勒雷达和 S 波段多普勒雷达进行加权融合，给予 **X 波段 0.6 的权重**，**S 波段 0.4 的权重**，解决单雷达覆盖不足的问题。

$$\begin{cases} U = w_x * U_x + w_s * U_s \\ V = w_x * V_x + w_s * V_s \end{cases}$$

其中 V_x 为 X 波段反演得到的东西向风速， U_x 为 X 波段反演得到的南北向风速， U_s 为 S 波段反演得到的东西向风速， V_s 为 S 波段反演得到的南北向风速。 w_x 为 X 波段的权重，应为 0.6， w_s 为 S 波段的权重，应为 0.4。通过加权分配不同的比重构建背景场，有优势在于综合 X 波段分辨率高，S 波段穿透性强，高度保持了**风场物理的一致性**，为接下来的最优插值提供了高质量的背景场。

(3) 集合卡尔曼滤波简化方法插值

最优插值(OI)是集合卡尔曼滤波的简化方法，是一种最小二乘估计法，通过求解使分析误差方差：

$$J(x_a) = (x_a - x_b)^T B^{-1} (x_a - x_b) + (y - Hx_a)^T R^{-1} (y - Hx_a)$$

最小的权重，得到最优解 x_a 。本文将多普勒雷达重建的水平风场加权融合后的背景场设为 x_b ，观测数据为 y_0 ，风廓线数据以及地面气象站数据都属于观测数据，其中风廓线数据负责提供不同高度上的风场，即提供东西向、南北向风场分量 u 、 v ，地面气象站提供近地面的风场，充分弥补由 X 波和 S 波构建的背景场在垂直方向上数据不足的缺点。最优解 x_a 公式为：

$$x_a = x_b + K(y_0 - H(x_b))$$

其中 H 是一个通过**三维空间插值**将背景场从网格区域映射到观测空间的**观测算子**，即相应映射到风廓线的观测位置和高度。在地面气象站数据处理中，本文**反距离加权法(IDW)**，将地面离散的气象站点数据插值到背景场的均匀网格区域中，提取近地面的 u 、 v 风量数据，在风廓线数据的处理中，根据风廓线的站点的经纬度数据确定站点在背景场中的水平位置，将风廓线数据的垂直层使用**三维样条插值**对缺失值进行插值，扩充数据点，与背景场垂直层高度进行匹配，从而实现三维线性插值，完成风廓数据的三维数据同化。 K 是根据背景场和观测场误差协方差计算得到的**增益矩阵**，又叫做权重矩阵。增益矩阵 K 的计算：

$$K = BH^T (HBH^T + R)^{-1}$$

其中：

B :背景误差协方差矩阵。

R :观测误差协方差矩阵。

H^T :观测算子的转置。假设背景误差和观测误差相互独立，互不相关，且均值为 0，

都服从高斯分布。

背景误差协方差 B 使用高斯函数模型：

$$B_{ij} = \sigma_b^2 \exp\left(-\frac{\|r_i - r_j\|^2}{2L^2}\right)$$

其中： σ_b 为背景误差标准差； L 为相关尺度。

观测误差协方差 R 设为对角矩阵，分别代表风廓线和地面站。

$$R_{kk} = \sigma_{o,k}^2, \quad \sigma_o = \begin{cases} 0.3 \text{ m/s} \\ 0.8 \text{ m/s} \end{cases}$$

6.2 涡旋耗散率计算湍流强度

湍流的空间统计特征可通过空间结构函数描述。纵向二阶结构函数的定义为：

$$D_{LL}(\vec{p}; \vec{s}) = \langle [v_L(\vec{p}) - v_L(\vec{p} + \vec{s})]^2 \rangle$$

其中， $v_L(\vec{p})$ 为位置 $\vec{p} = (x_0, y_0, z_0)$ 沿位移矢量 $\vec{s} = (x, y, z)$ 方向的速度分量， $\langle \rangle$ 表示集合平均。

充分发展的三维各向同性湍流的统计特征可通过柯尔莫哥洛夫模型很好地描述，柯尔莫哥洛夫模型是一个描述充分发展湍流在小尺度上统计规律的普适性理论。其核心是揭示了湍流能量如何从大尺度向小尺度传递，并最终被耗散的过程，并精准预言了其中惯性子区间内的-5/3 能谱律。

此时以涡旋耗散率 ϵ 表征的湍流强度与 $D_{LL}(\vec{p}; \vec{s})$ 的关系为：

$$D_{LL}(\vec{p}; \vec{s}) = C_k \epsilon^{2/3} s^{2/3}$$

其中 $C_k \approx 2$ 为 Kolmogorov 常数， $s = |\vec{s}|$ 为分离距离。

因此由上述公式我们推算得到涡旋耗散率 ϵ 的公式为：

$$\epsilon \approx \left(\frac{D_{LL}(s)}{C_k \cdot s^{2/3}} \right)^{3/2}$$

湍动能耗散率 ϵ 是描述湍流能量从平均流动中被产生并最终通过粘性作用耗散成热能的关键物理量，单位是 m^2/s^3 。

湍流强度的定义式为：

$$TI = \frac{\sigma_u}{U} \times 100\%$$

其中 σ_u 是风速波动的标准差， U 是平均风速，本文使用涡旋耗散率间接估算湍流强度 $TI = \epsilon^{1/3}$ 。

6.3 基于多角度分析三维空间湍流强度计算模型的准确性

6.3.1 三维湍流强度、三维湍流动能空间分布研究

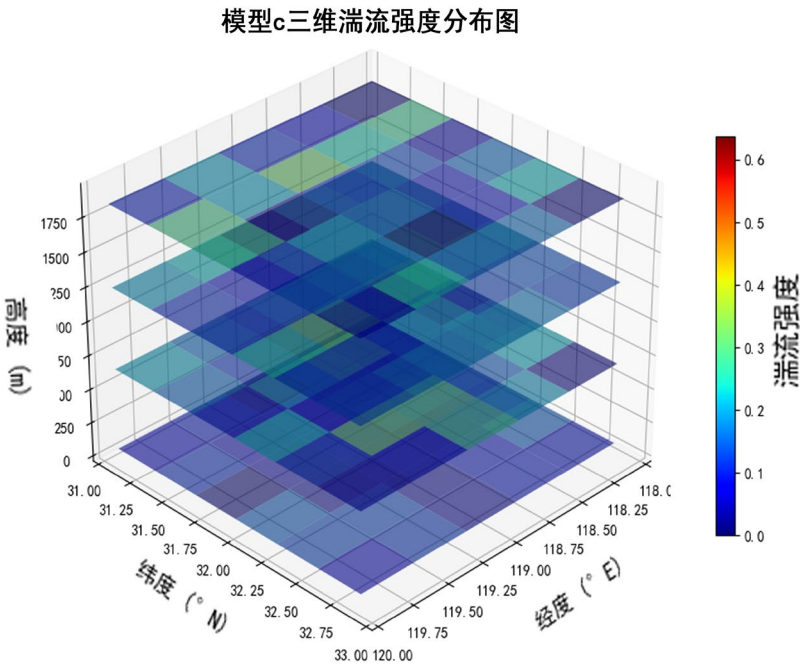


图 15 模型 c 三维湍流强度空间分布图

由图 15 可知，湍流强度在**地理空间上分布不均**，垂直结构上存在显著的分层特征，不同高度，湍流强度的平均大小也不同，由 0m 到 500m 湍流强度**递进增加**，这表明在**低空范围**由于近地面的地理空间上的复杂多样性，导致空气层**气流不稳定**，极易造成湍流的出现。水平方向上，色彩分布存在不均匀的斑块状结构，表明即使高度层相同，但由于其他因素的影响，同一高度层大部分地区稳定，个别地方也会存在湍流，为最优航路规划奠定前提条件。

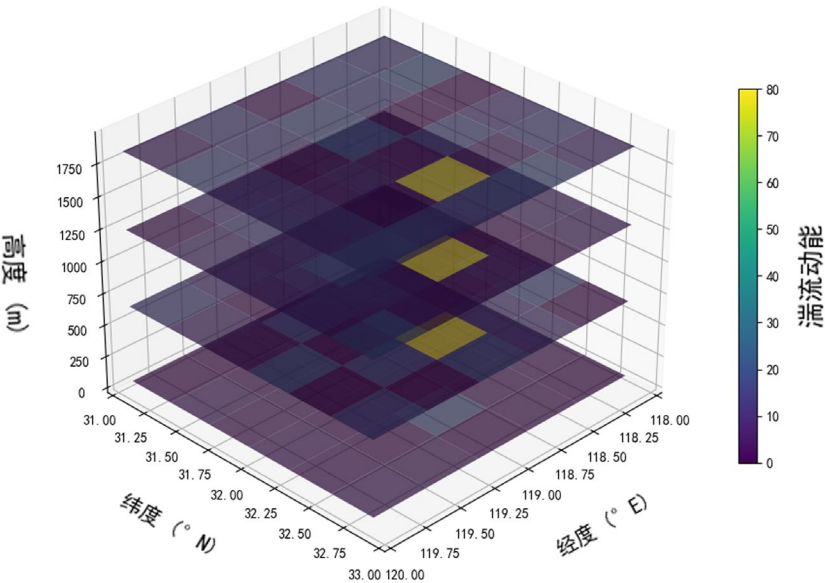


图 16 三维湍流动能空间分布图

对比三维湍流强度图、三维湍流动能图，结合两种湍流指标，相互交叉验证，对比分析，确保模型 c 构建的正确性。当高度从 0m 到 500m 时，湍流指标大小同样是总小到大，在 1000m 左右又有所降低，这是由于随着高度的增加，地表摩擦减小，大气结构趋于稳定，湍流受到抑制。

但 TKE 数值的计算，存在缺陷，导致异常出现个别极大值，这将成为后续优化数据处理的主要部分。

6.3.2 基于地理信息空间湍流强度研究

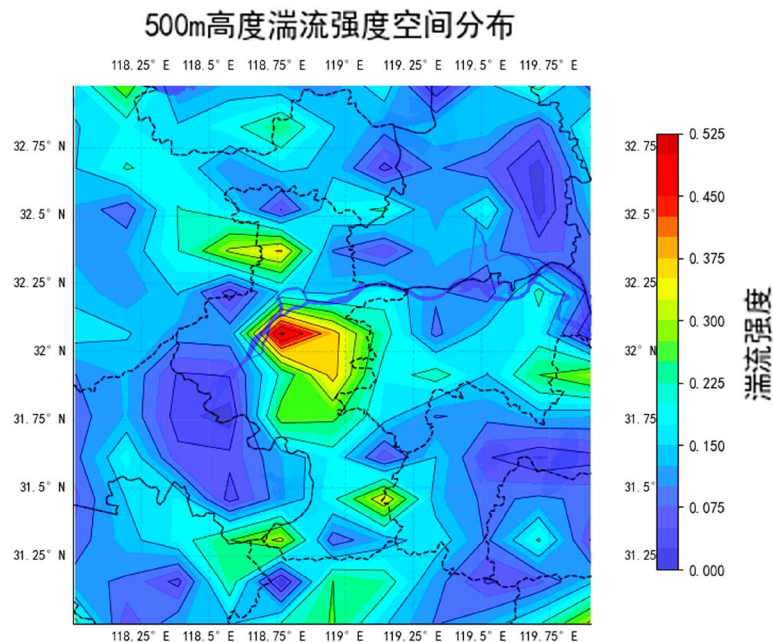


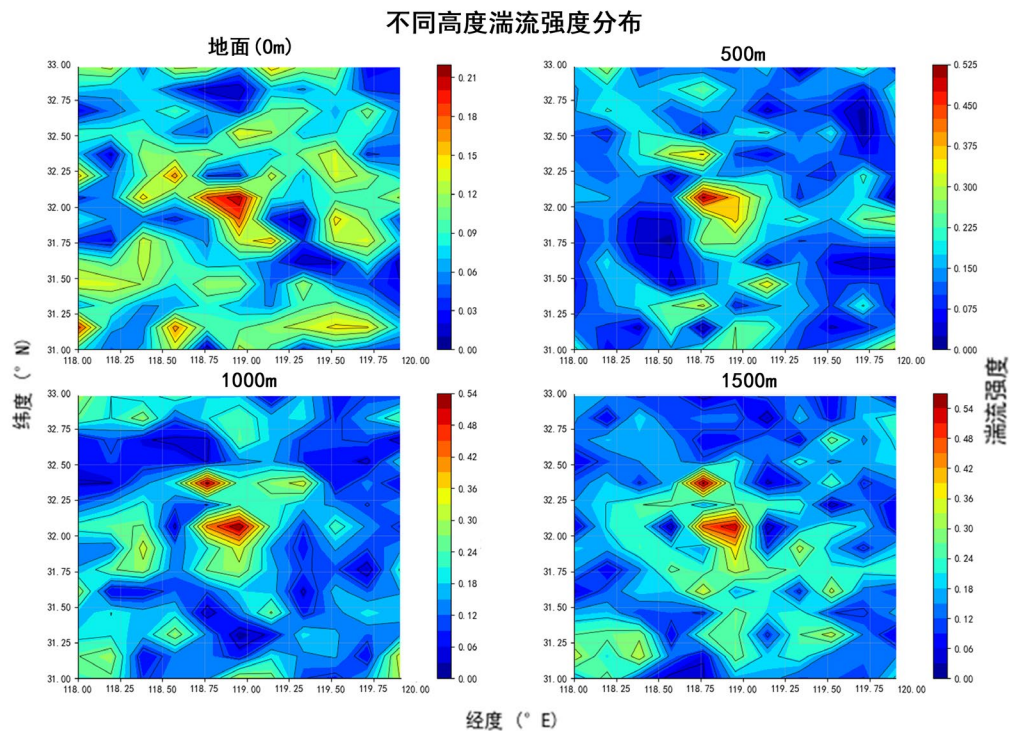
图 17 三维湍流动能空间分布图

上述为 500m 高度湍流强度分布图，包含行政区域的边界，即黑色虚线为市区的边界线，此外还将长江流域进行可视化，凸显该区域的河流特征，进一步精细分析影响湍流强度的地形因素。

由此热力图可得知，在河流流经区域湍流强度较低，大约在 0.075 以下，空气层结构稳定，湍流存在概率较低。

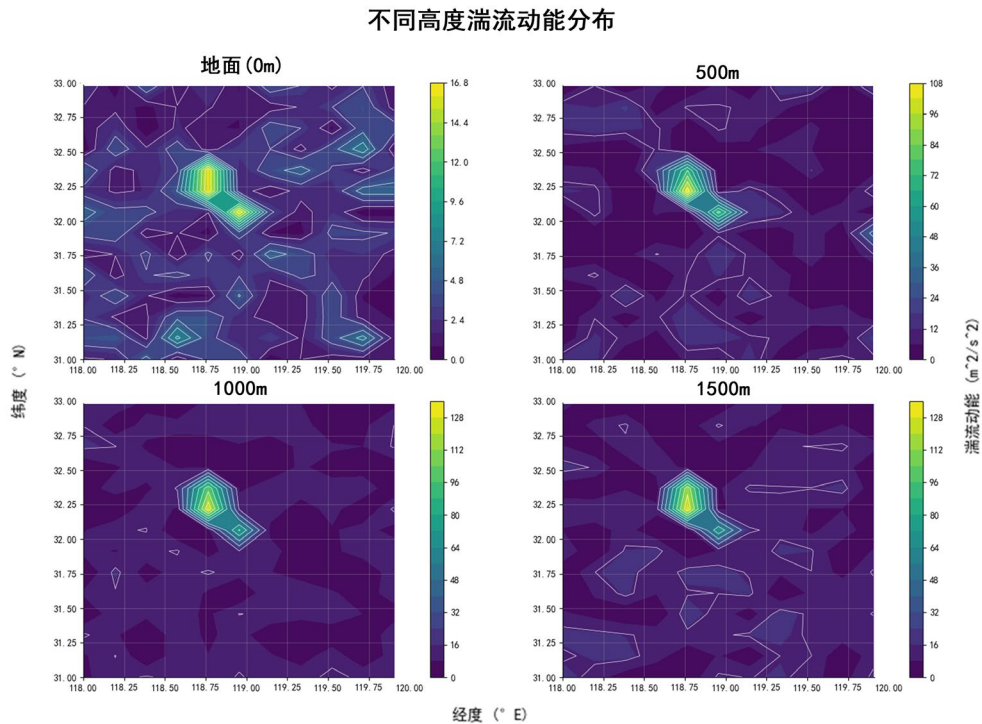
而在城市地界，湍流强度较高，大约在 0.225 以上，这是由于城市高空建筑数量多，地形复杂，并伴有飞机等飞行器的影响，导致湍流强度较高，容易出现湍流。

(1) 基于不同高度层的湍流强度垂直结构分析



由不同高度层的湍流强度分布图，可以得知本论文所讨论的区域范围内，在中心点处存在较强的湍流，往四周扩散湍流强度相对应的有所降低，从高度层多样性的角度体现了模型 c 的可靠性，为下文从不同高度对比湍流强度以及湍流动能的分布奠定基础。

(2) 基于不同高度层的湍流强度垂直结构分析



湍流强度与湍流动能相互验证图的分析是评估湍流特征一致性的重要手段，上面的二者的空间分布对比展现了湍流的区域性特征。两种指标在空间格局上表现出高度相似性，体现了 c 模型通过高度层统计和区域聚焦满足了题目所要求范围内的精确研究，湍流强度和 TKE 的相互验证为模型可靠性评估提供了重要手段。

(3) 针对南京市湍流强度、湍流动能垂直分布结构研究

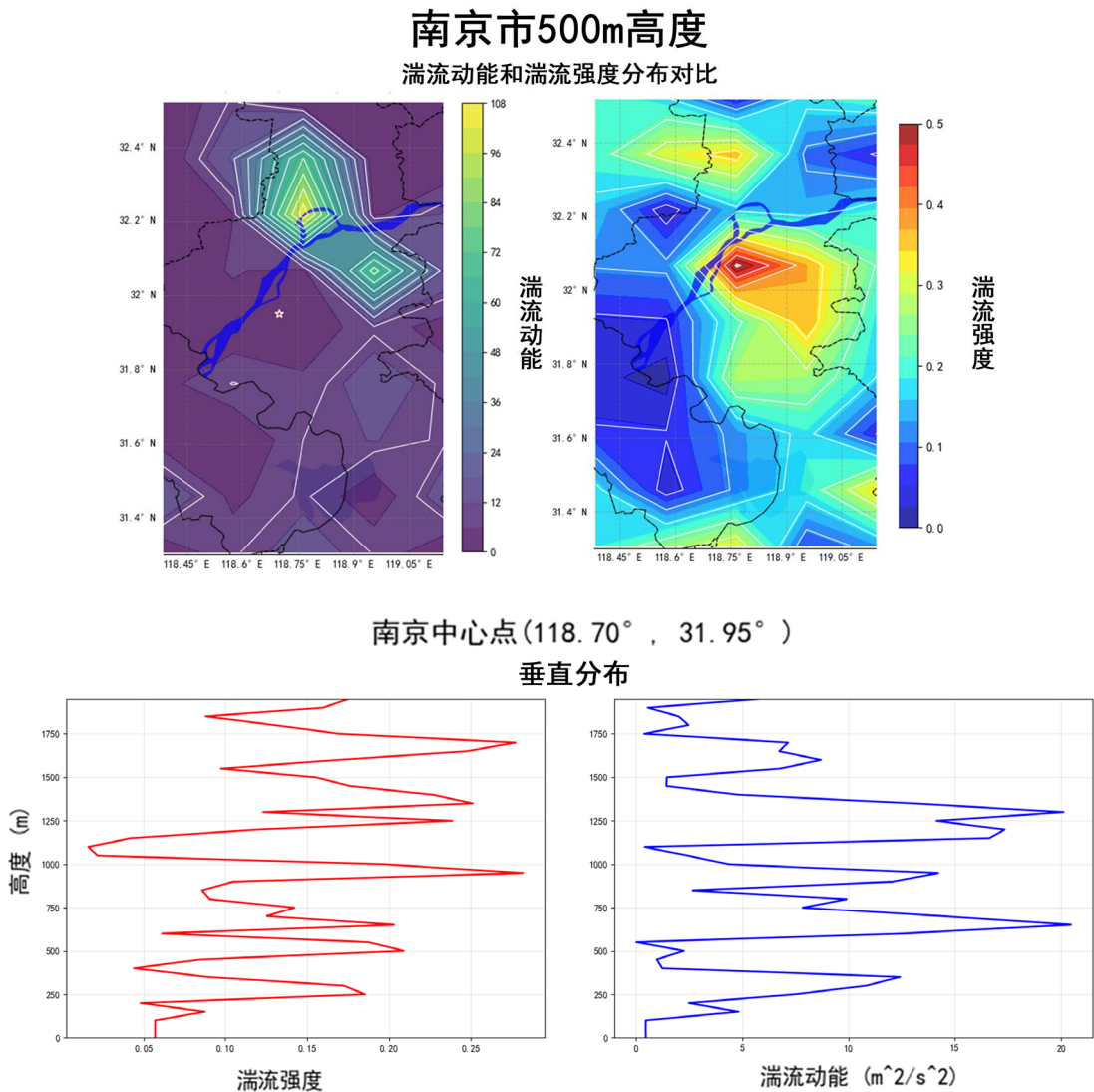


图 20 南京市空间湍流动能强度垂直分布图

湍流强度的垂直分布图变相反映了风速的强弱，曲线在低空大约 1000m 以下集中较为密集，表明风速变化相对平缓，随着高度上升，曲线剧烈波动且显著发散，尤其在 1000 米至 1500 米之间，湍流强度值分布范围扩大，清晰地表明在该高度范围存在一个湍流的活跃层，这通常与上空的大气结构，或强对流有关。

湍流动能垂直分布图表明，在近地面至 1000 米左右维持较高值，这与地表摩擦有较大的关联性，随后在 1250 米至 1500 米以上呈现出明显的衰减趋势，表明湍流活动在此高度以上显著减弱，大气变得相对平稳。总结南京上空大气在约 1000 米至 1250 米的高度存

在一个显著的湍流界面。其下，湍流活动和湍流动能都较为充沛；其上，大气层结趋于稳定，湍流迅速减弱。两张图相互印证，勾勒出南京地区大气边界层的垂直分布。

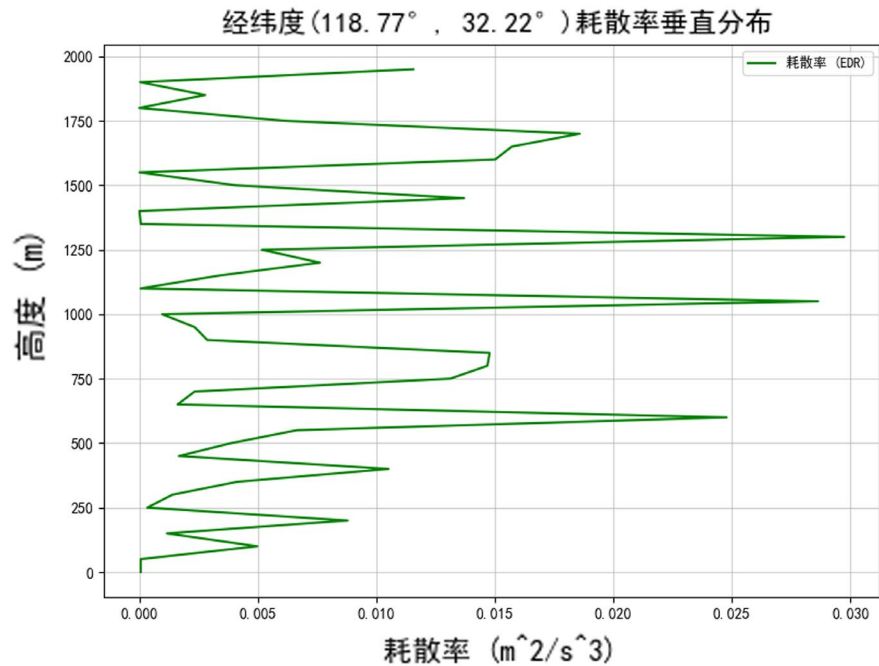


图 21 耗散率垂直分布图

本文构建的 c 模型通过**多维度可视化手段**系统揭示了地面气象站覆盖区域内低空湍流场的空间分布与垂直结构特征。综合分析表明，该区域湍流场呈现显著的垂直分层特性：在 **750-1250 米** 高度区间存在**持续的强湍流层**，对于低空飞行器来说存在潜在的颠簸风险。1500 米以上高度湍流强度明显衰减，指示大气边界层顶位置，为高空飞行提供相对平稳的空域。三维可视化进一步直观展示了湍流强度在经纬度-高度空间中的复杂分布模式。

七、问题三：基于多尺度数据的低空湍流场及航路规划策略研究

由于低空区域湍流存在的概率较大，为保障低空飞行器的安全，需要规划正确的航路避开湍流，因此本文基于预报天气场和雷达站等有关数据，去建立一种湍流强度的场，并且用于最优航路的预测，使用两个模型来进行最优航迹规划。

7.1 基于预报天气的湍流强度场最优航路规划

该模型为模型 d，通过物理诊断或统计方法改造尺度较大的气象场为精度较高的湍流强度场，计算 TKE 作为湍流强度的度量，将监测能力延伸至预报情形，并以模型 c 的分析结果作为验证基准进行优化，其最终输出直接服务于规避湍流程度高的区域的最优航路规划。

7.1.1 基于数值天气预报数据湍流强度场的构建

·风场数据文件结构分析

以问题二给的“wrfvar_output_1km.202507310200.nanjing.nc”文件为例，其数据结构如下表所示。

表 2 数值天气预报数据结构表

变量名称	因素类型
ACGRDFLX	Time,south_north,west_east
ACHFX	Time,south_north,west_east
ACLHF	Time,south_north,west_east
ACSNOM	Time,south_north,west_east
ALBBCK	Time,south_north,west_east
ALBEDO	Time,south_north,west_east
.....	
Pres	Time,bottom_top,south_north,west_east
Z	Time,bottom_top,south_north,west_east
RH	Time,bottom_top,south_north,west_east

其中，构建模型 d 需要东西向风速分量 U_a ，南北向风速分量 V_a ，垂直风速分量 W_a ，位势高度 Z 等变量，本文对风速分量进行逐点计算，并依据上述公式计算 TKE

·标准尺度新型湍流强度场的生成

本文根据需求规定的分辨率水平，生成新的以 TKE 代表湍流强度的新的场，即将原始的风场数据插值到统一规则的空间网格上，并根据地面的地理区域，截取目标区域的数据，为包含所有地面自动气象站的矩形区域。

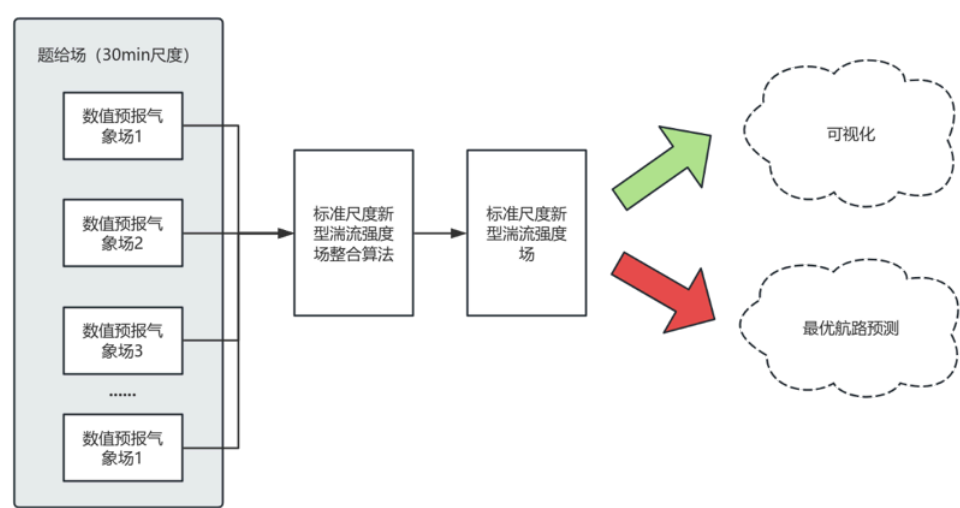


图 22 标准尺度新型湍流强度场的生成流程图

新建 TKE 场的几个关键维度和变量，首先是维度方面， z 为高度层变量，有 41 个值，从 0 米到 2000 米，代表不同的高度层； lat 代表纬度，范围从 31.06 到 32.73 度，大约覆盖了地面气象站的所在区域， lon 代表经度，范围从 118.2 到 119.4 度，大约覆盖了地面气象站的所在区域；其次在数据变量方面， $time$ 代表时间，表示该场是有一个时间维度的，说

明不同时间点上的湍流动能数据，即该 TKE 场的湍流强度值是可以随着时间更新的；最重要的，TKE 变量，这是主要的物理量，表示在不同的高度、纬度和经度上的湍流动能大小，为该 NC 文件的核心内容。

显然，得出的 TKE 场满足题目要求。其高度 2km 以下、垂直分辨率 50m、水平分辨率 100m 的精度要求，经过对地面站数据的分析，我们最终决定建立一个纬度范围在 31-33° 之间，经度范围在 118-120° 之间且高度层位于 0-2000m 的一个高分辨率湍流强度场。

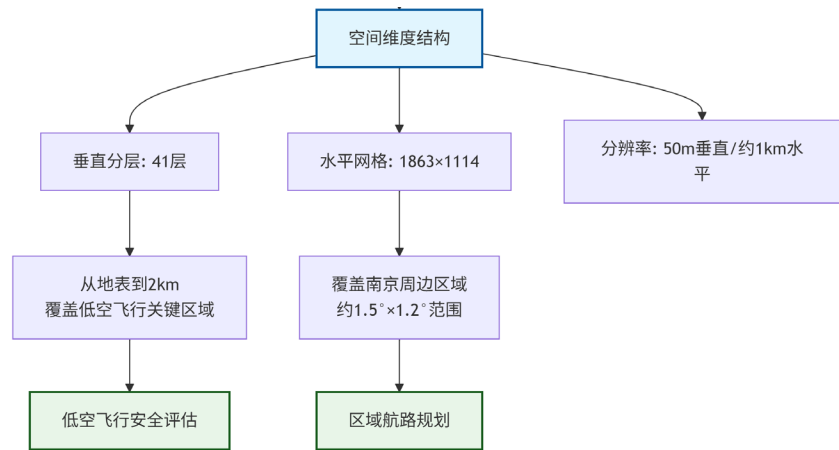


图 23 标准尺度新型湍流强度场空间维度结构图

·标准尺度新型湍流强度场可视化

热力图是一种用于表示数据值大小和差异的二维图表，通过不同的颜色和色度标准来展示数据的分布情况。常用于展示数值型数据在时空维度的变化，也可用于展示密度、温度、湍流强度等物理量，热力图的颜色范围通常从冷色到暖色，表示从低到高的变化，能够直观地反映出数据的空间分布、趋势及异常区域。

本文用热力图展示 2025 年 7 月 31 日 02:00 时刻的湍流动能（TKE）空间分布情况。图的布局是一个 4×5 的网格，每个小图代表不同高度层下的 TKE 空间分布，每个小图上方有标注，代表不同的高度层。图中也使用了不同颜色来表示 TKE 的值的大小，颜色从低值到高值表示湍流动能的强度。颜色条展示了 TKE 的具体数值范围,越红的区域表示湍流强度越高，而蓝色则表示湍流较弱。

可以通过热力图的形式直观的感受这个场。

图 24 显示的是 2025 年 7 月 31 日凌晨 2 时目标区域的湍流强度，由蓝到红为湍流强度由弱到强，可以看出中间部分湍流强度较小，但某些部分的湍流强度高，如图的右上角部分，存在明显的急流，不利于飞行器飞行。

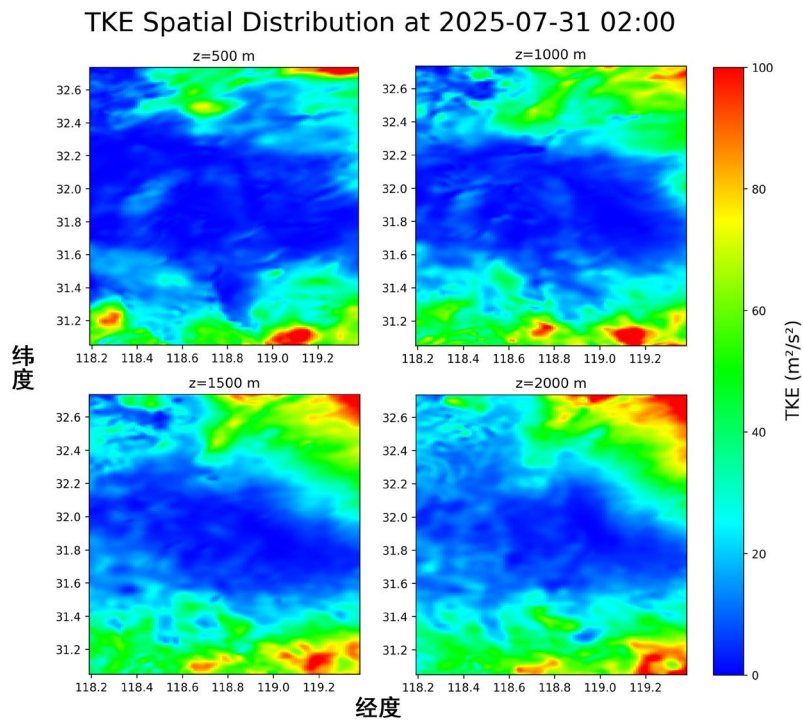


图 24 2025 年 7 月 31 日凌晨 2 时目标区域的湍流强度图

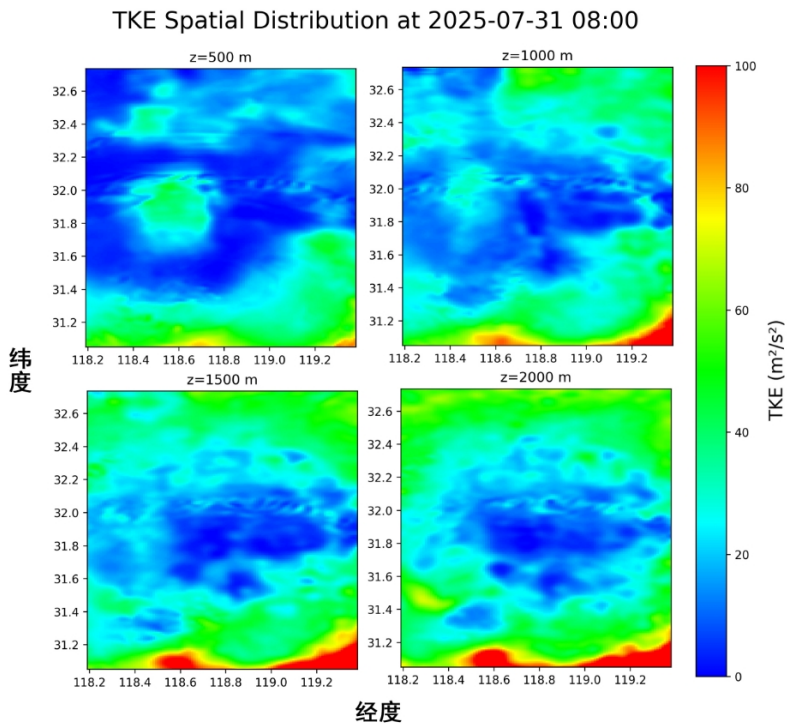


图 25 2025 年 7 月 31 日 8 时目标区域的湍流强度图

这两张图的对比表明，湍流强度随时间的推移发生了明显变化，从较为平稳的状态逐步转向较为剧烈的湍流活动，为下一步的飞行路径规划提供了重要的参考价值。如图 25 所示，可以看到早上 8 时的湍流强度更为复杂，对于规划出最优航路带来挑战。

本文提供三维的视角如图 26 所示，可以直观地观察这个 TKE，我们取 7 月 31 日凌晨 2 时的 TKE 场为例：

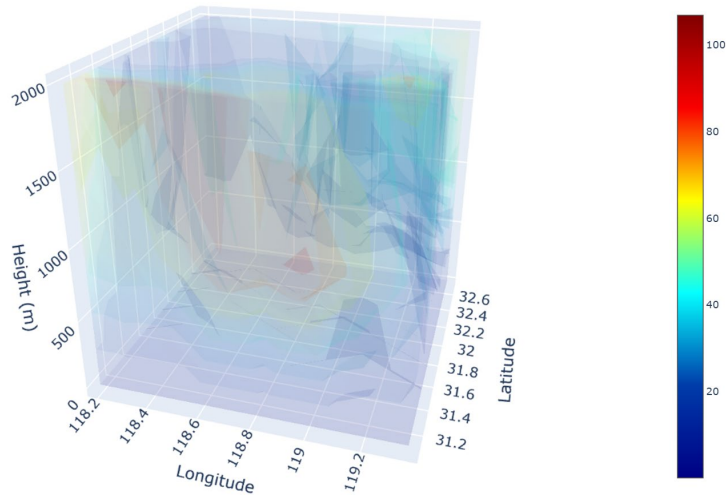


图 26 2025 年 7 月 31 日 2 时目标区域的三维湍流强度图

如图 27 所示，明显地，可以观察到这个区域内存在一个较强的湍流空间，如图中的红色部分所示区域，同时呈现放上 7 月 31 日早晨 8 时的 TKE 场的 3D 图进行对比。

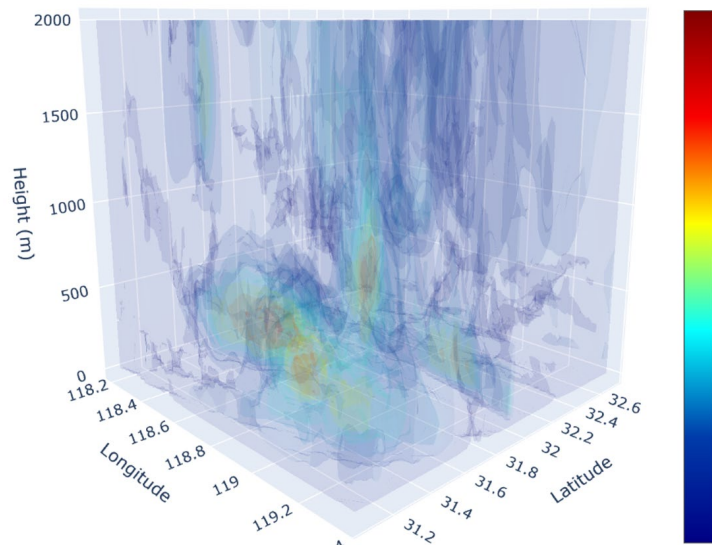


图 27 2025 年 7 月 31 日 8 时目标区域的三维湍流强度图

7.1.2 基于标准尺度新型湍流强度场的最优航路预测

对于目标进行定义，给定一个起点和终点来进行路径计算，设立距离，湍流强度代价函数，给定飞行器运行速度，以及时间限制作为约束条件，通过最小化代价值来优化目标算法。

(1) 最优航路预测原理

·坐标转化

经度方向每度米数：

$$m_{lon} = 111320 \times \cos(lat_{mean})$$

纬度方向每度米数：

$$m_{lat} = 111132$$

转换公式为：

$$\begin{cases} x = (\text{lon} - \text{lon}_0) \times m_{lon} \\ y = (\text{lat} - \text{lat}_0) \times m_{lat} \end{cases}$$

其中 $(\text{lon}_0, \text{lat}_0)$ 为起始经纬度， lat_{mean} 为起点经纬度为两点纬度平均值。

我们接下来还需要定义路径代价计算公式，作为约束飞行的条件，我们规定路径代价包括了距离代价和 TKE 湍流惩罚代价：

$$\text{cost} = \sum_{i=1}^n \left[d_i \times (1 + \alpha \times \text{TKE}_i^\beta) \right]$$

其中， d_i 代表第 i 段路径的距离， TKE_i 代表第 i 段路径的湍流动能值， α 为 TKE 权重系数， β 为 TKE 幂次系数。

对于总的思路而言，首先，我们需要进行数据加载与预处理，读取文件中的 TKE 数据、经纬度网格和高度数据，将经纬度坐标转换为以米为单位坐标，便于后续航行距离的计算。然后我们规定出最优路径为 A^* ，同时，我们也随机的生成一些直线路径，即由起点到终点的线性插值路径，同时我们也规划出一些利用三次样条插值生成多条随机路径，计算所有路径的总代价，以验证路径 A^* 的合理性。

当然，我们给出约束条件。

$$\text{s.t.} \begin{cases} \text{1. 空间约束} \\ \begin{cases} \text{lon}_{\min} \leq \text{lon} \leq \text{lon}_{\max} \\ \text{lat}_{\min} \leq \text{lat} \leq \text{lat}_{\max} \\ \text{alt}_{\min} \leq \text{alt} \leq \text{alt}_{\max} \end{cases} \\ \text{边界检查功能} \\ \text{2. 湍流约束} \\ \begin{cases} \text{TKE} \leq \text{TKE}_{\max} \\ \text{cost}_{\text{tke}} = \alpha \times \text{TKE}^\beta \times d \end{cases} \\ \text{湍流检查功能} \end{cases}$$

其中， d 为路径段距离，用以防止飞行器逃逸或者无法有效避开湍流。

接下来，我们需要输入数据，包括 TKE 场数据，风速数据，飞行器速度等参数，来进行数据分析，我们有初始条件：

$$\begin{cases} \text{lat}_0 = 31.05, \text{lon}_0 = 119.33, \text{alt}_0 = 400.0 \\ \text{lat}_g = 32.73, \text{lon}_g = 119.40, \text{alt}_g = 1800.0 \\ v = 30.0(\text{m/s}) \\ t_{\max} = 3.0(\text{h}) \\ \alpha = 200.0 \\ \beta = 1.5 \end{cases}$$

其中：

$\text{lat}_0, \text{lon}_0, \text{alt}_0$ 分别为起点纬度、经度、高度，

$\text{lat}_g, \text{lon}_g, \text{alt}_g$ 分别为终点纬度、经度、高度，

v 为飞行器速度，

$t_{\max}$ 为最大飞行时间限制，

α 为 TKE 权重系数，

β 为 TKE 幂次系数。

然后，我们规划目标与优化函数，对于路程，目标是最小化航程所需时间，或者最大化飞行效率；对于安全，我们应避免飞行器进入高风险区域，综合而言，我们要使得路径的路径代价最小，即要求：

$$\text{cost}(A^*) = \min \left\{ \sum_{i=1}^n \left[d_i \times (1 + \alpha \times \text{TKE}_i^\beta) \right] \right\}$$

这样的路径 A^* 就是我们理想的最优航径。

主要的步骤如下面的流程图所示。

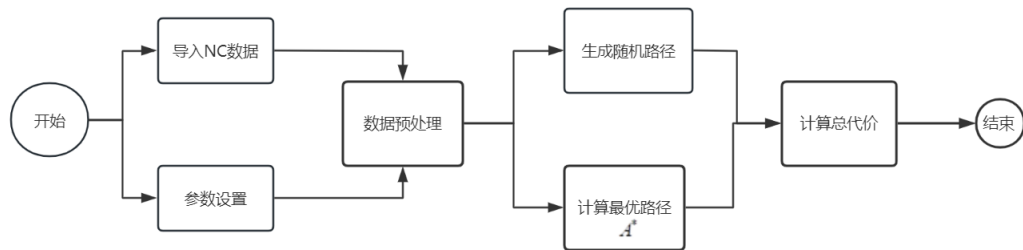


图 28 模型 d 最优航路规划算法流程图

(2) 最优航路合理性分析

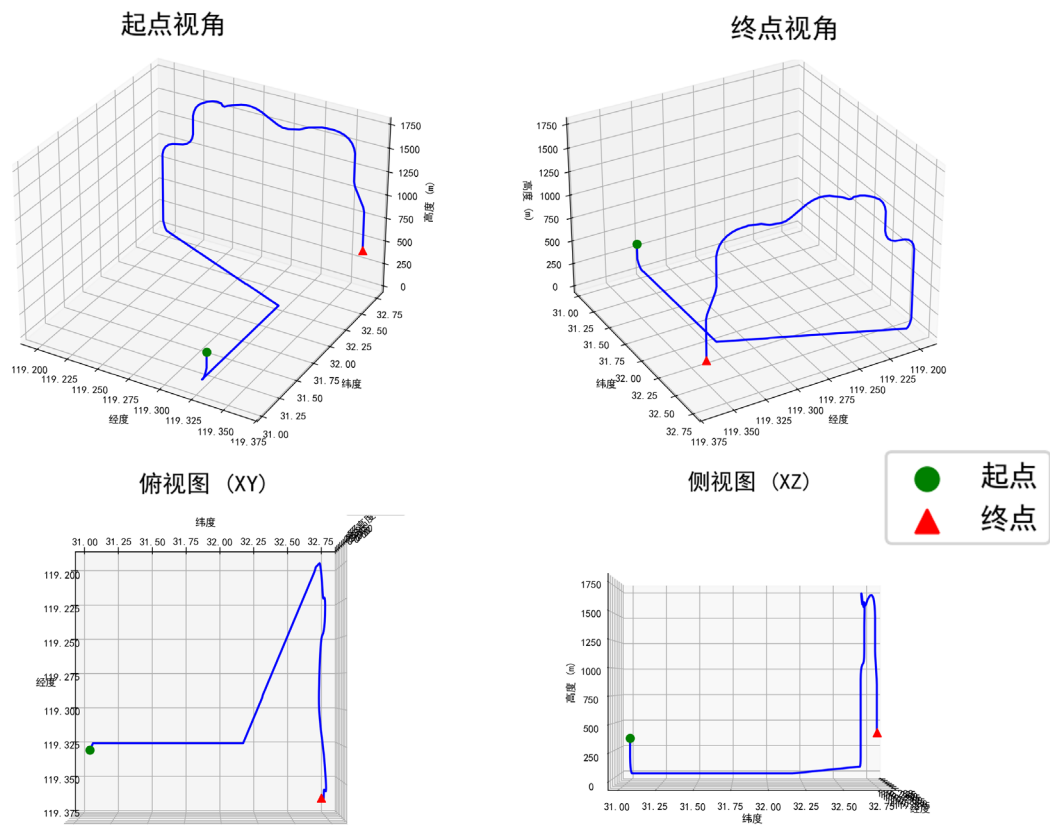


图 29 模型 d 最优航路三维展示图

由图 29 中可知，在规定的三维空间中，航路有明显的方向变化，并非沿单一直线运行，表明飞行器考虑了立体空间内的湍流强度而构造的复杂航路；另外，从部分视图能看看到，航路存在较大幅度的高度升降，不是保持恒定高度飞行，这也是牺牲了路径代价来减少湍流代价的表现之一。整体航路呈现一定的走向规律，经过了约束条件限制，权衡了湍流和时间效率。

本文截取了飞行器运行几个时间的快照，也体现了飞行器想要避开湍流区域的倾向，下图为每个半小时截取一次的飞行器运行快照

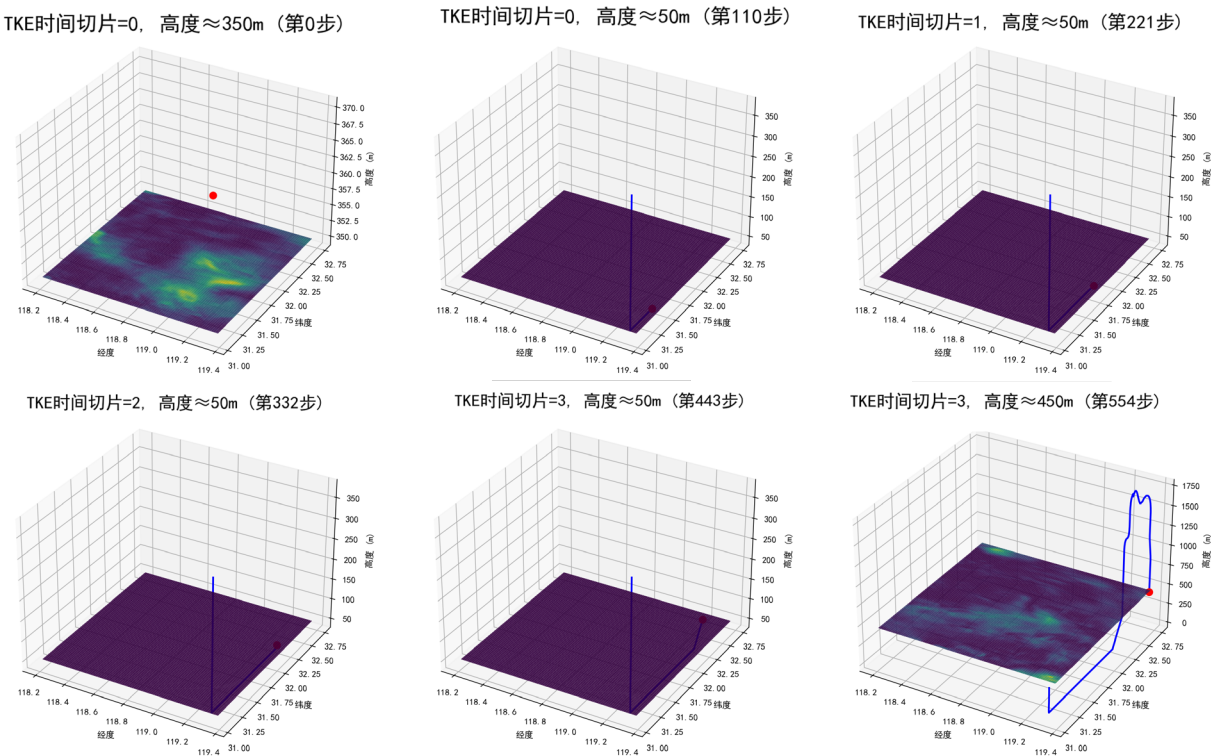


图 30 模型 d 最优航路运行快照

为了体现 A^* 的最优性，本文计算了几种不同航线的代价，包括直接直线通过以及特定的曲线通过，结果如下表所示。

表 3 标准尺度新型湍流强度场的生成流程图

路径类型	总代价 cost
A^* 最优路径	1.14×10^7
直线路径	4.2×10^8
随机路径 1	4.55×10^8
随机路径 2	5.77×10^8
随机路径 3	1.04×10^9
随机路径 4	6.92×10^8
随机路径 5	$5.4E \times 10^8$
随机路径 6	1.14×10^9
随机路径 7	9.27×10^8
随机路径 8	5.08×10^8
随机路径 9	1.04×10^9
随机路径 10	1.08×10^9

明显地，其他的随机路径的总代价已经接近**亿级**单位，而路径 A^* 的总代价则是**百万级**，明显路径 A^* 是这几个**路径最优的**，由于无法展示所有随机路径， A^* 实际上是所有路径可能中总代价最小的最优航路了。

本文在此展示部分航路数据：

表 4 最优航迹运行过程表

步骤	经度	纬度	高度（m）	时间（s）
0	119.33	31.05	400.0	-
1	119.33	31.05	370.0	0
2	119.33	31.05	340.0	1. 6667
3	119.33	31.05537	250	5
4	119.33	31.05537	200	6.6667
5	119.3291	31.0561	150	8.3333
6	119.3283	31.05754	110	10
...	...	...	...	...
549	119.364	32.7284	650	7101.49
550	119.364	32.7284	600	7103.16
551	119.364	32.7284	550	7104.82
552	119.364	32.7284	500	7106.49
553	119.364	32.7284	460	7108.16
554	119.364	32.7284	430	7109.82
555	119.4	32.73	400	-

7.2 基于多种观测资料的湍流强度场最优航路规划

7.2.1 基于 LSTM 对风廓线雷达设备数据的湍流动能场预测模型构建

（1） 基于 LSTM 模型训练

构建基于 LSTM 对三维空间湍流强度的预测模型，过程如下。首先，我们对风廓线雷达的三个站点的数据进行整合处理，建立三维网格，对每个网格点 02 点-05 点间的数据使用 LSTM 模型训练，得预测到 05 点-06 点的三维空间湍流强度。LSTM 训练过程中的训练损失和验证损失随迭代次数的变化情况如下图所示。

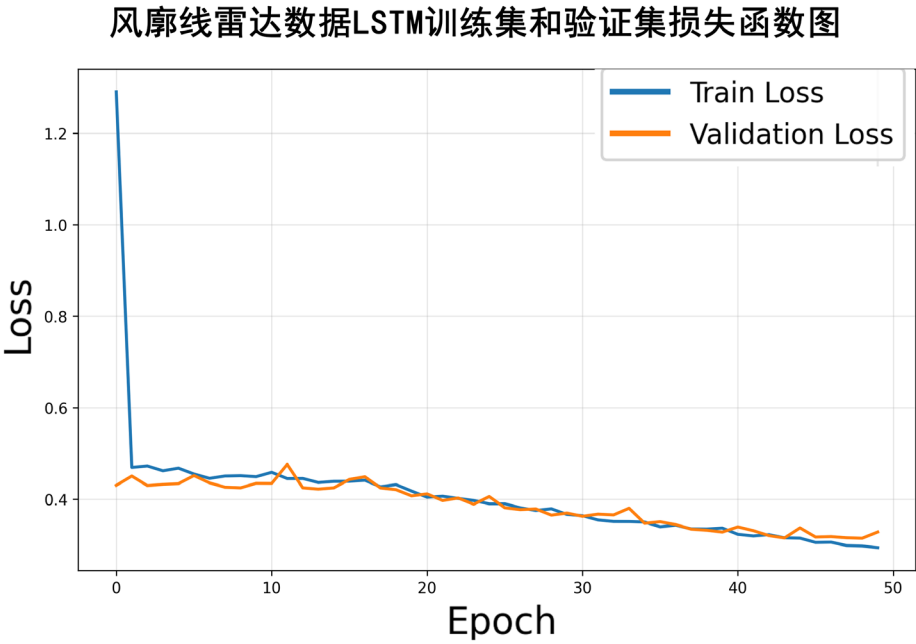


图 31 风廓线雷达数据 LSTM 训练集和验证集损失函数图。

(2) 模型验证优化

对所给用于验证的数据中 05 点-06 点风廓线雷达三个站点的数据真实值进行同样的三维网格建立，匹配所建立模型的预测数据与由真实值所得的验证数据的三维坐标，即可验证模型的可靠程度。风廓线雷达 LSTM 模型的 05 点-06 点的三维空间湍流强度预测结果与对比图如下。

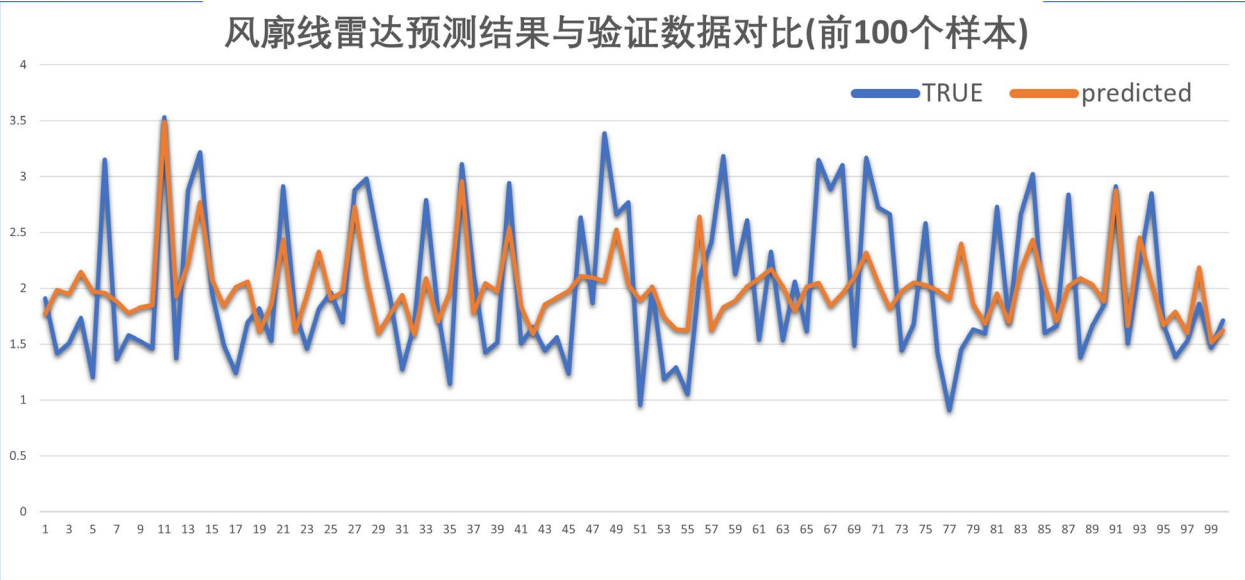


图 32 前 100 个样本的风廓线雷达预测结果与验证对比。

拟合优度为 0.27，显然模型可靠性不高，因此我们需要调整 LSTM 训练过程参数来减少误差，但经过参数优化后拟合优度 0.28 仍然不高，说明仅靠风廓线雷达预测湍流强度并不可靠，需要结合其他设备数据进行融合分析，来提高模型对湍流强度的灵敏度，所以下面引入 S 波段天气雷达。

7.2.2 基于 LSTM 对 S 波段天气雷达设备数据的湍流动能场预测模型构建

(1) 数据预处理

对得到的湍流强度场进行插值，常用于三维空间插值的方法有三维线性插值（3D Linear Interpolation）、三维径向基函数插值（3D Radial Basis Function, RBF）和克里金插值（Ordinary Kriging Interpolation）等插值方法。

基于前面问题分析结果湍流强度分布**不规则程度较高**，所以选用三维径向基函数插值，**插值精度高，平滑性好**，适用于高维散乱数据，可**处理任意分布的点**，以此来对未覆盖区域进行插值处理。

三维径向基函数插值原理是通过径向基函数构建全局插值模型，保证插值结果通过所有已知点，核心步骤如下：

假设我们有已知散乱点为 $\{x_1, x_2, \dots, x_n\}$ ，函数值为 $\{f_1, f_2, \dots, f_n\}$ ；

我们定义插值函数为：

$$f(x) = \sum_{i=1}^n \lambda_i \cdot \phi(\|x - x_i\|) + p(x)$$

其中 ϕ 是径向基函数， $p(x)$ 是低阶多项式，用于消除边界趋势；

然后，我们可以通过插值条件 $f(x_i) = f_i$ 求解系数 λ_i 和 $p(x)$ 的参数。

我们运用上述原理，进行三维径向基函数插值，能够得到该插值结果覆盖 2025-07-31 的 02:00 至 05:00 的时间段，空间范围为经度 118.0–120.0、纬度 31.0–33.0、高度 148.3–1999.3m，数值分布在 7.53–66.98 之间，**分辨率极高**，包含约 2.27×10^5 个经度点、 2.43×10^5 个纬度点和 1400 个高度层。

(2) 模型训练与预测

鉴于上述**巨大数据量**与 LSTM 的结构**复杂度**导致其**训练难度高**，训练时间较长，因此需要**降低网格密度**以优化 LSTM 训练速度，但由此带来的是精度上的损失。

对上述三维空间划分成 $10 \times 10 \times 5$ 的网格大小即水平区域上，用每格范围内的数据进行 **k-means 聚类**出代表值。

k-means 方法即在三维空间中，每个数据点表示为一个三维向量： $x_i = (x_i^1, x_i^2, x_i^3)$ ，其中上标 1,2,3 分别表示 x 、 y 、 z 三个维度。

距离的度量通常使用**欧氏距离**计算两点之间的距离：

$$d(x_i, x_j) = \sqrt{(x_i^1 - x_j^1)^2 + (x_i^2 - x_j^2)^2 + (x_i^3 - x_j^3)^2}$$

为了计算方便，实际计算中常用平方距离：

$$d^2(x_i, x_j) = (x_i^1 - x_j^1)^2 + (x_i^2 - x_j^2)^2 + (x_i^3 - x_j^3)^2$$

最后，k-means 的目标是最小化所有簇的总误差平方和：

$$J = \sum_{k=1}^K \sum_{i \in C_k} d^2(x_i, \mu_k)$$

我们的目的即是：

$$J^* = \min \left[\sum_{k=1}^K \sum_{i \in C_k} d^2(x_i, \mu_k) \right]。$$

其中： K 是簇的数量， C_k 是第 k 个簇中的所有数据点， μ_k 是第 k 个簇的质心。

在三维空间中,质心是簇内所有点在三个维度上的平均值:

$$\mu_k = \left(\frac{1}{|C_k|} \sum_{i \in C_k} x_i^1, \frac{1}{|C_k|} \sum_{i \in C_k} x_i^2, \frac{1}{|C_k|} \sum_{i \in C_k} x_i^3 \right)$$

其中 $|C_k|$ 是第 k 个簇中数据点的数量。

经过迭代计算，对 k-means 聚类结果后进行 LSTM 训练，得出了基于 S 波段天气雷达数据的 LSTM 模型，模型损失函数图如下

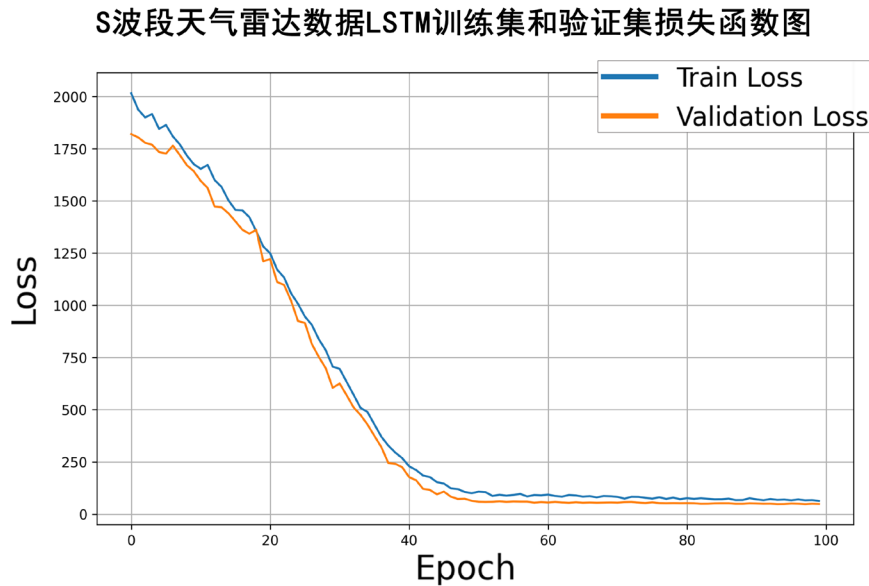


图 33 S 波段天气雷达数据 LSTM 训练集和验证集损失函数图.

用训练好的模型对 05-06 点进行预测得到新的湍流强度三维分布场。拟合优度达 **0.77**，S 波段雷达 LSTM 模型的 05 点-06 点的三维空间湍流强度预测结果与真实值对比图如下。

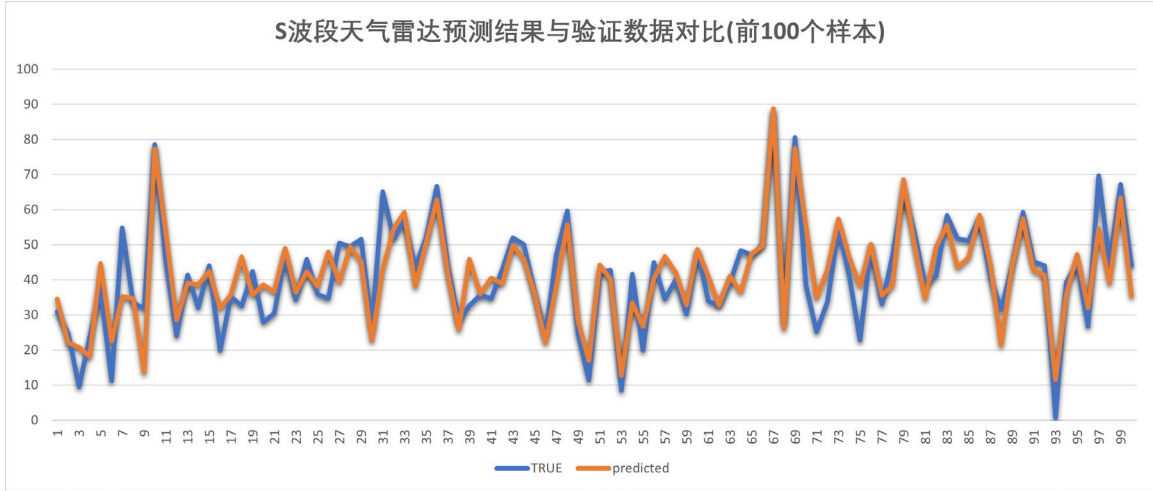


图 34 S 波段天气雷达预测结果与验证数据对比图

7.2.3 基于模型加权的湍流动能场最优航路规划

(1) 加权湍流动能场构建

参考文献通过 AIC、BIC 和残差矩阵值分配权重算出加权后模型。

我们先给出核心概念与前置定义，首先明确涉及的关键变量与物理意义，为后续推导奠定基础，我们的核心目标是使得让性能更优的湍流强度获得更高权重，同时通过归一化避免单一样本权重过大，实现全局平衡。

首先，原始性能指标存在局限性。我们若直接用 R_k^2 作为权重，存在两个问题，第一个问题， $\sum_{k=1}^n R_k^2 \neq 1$ ，不满足权重的基本性质，第二个问题，若某一湍流强度的 R_k^2 远大于其他样本，直接用 R^2 会导致权重极度倾斜，无法平衡湍流强度的贡献。

然后，我们进行归一化处理，通过构建映射关系，将 R_k^2 压缩到 $[0,1]$ 区间且总和为 1，本质是计算每个 R_k^2 在所有湍流强度总和中的占比，并以此作为权重。数学公式如下：

$$w_k = \frac{R_k^2}{\sum_{i=1}^n R_i^2} \quad (k=1,2,\dots,n)$$

其中，分子 R_k^2 体现第 k 个湍流强度的优先级—— R_k^2 越大，说明模型对该湍流强度的拟合越准确，应分配更高的权重，分母 $\sum_{i=1}^n R_i^2$ 为所有湍流强度的性能总和，作为归一化因子，确保最终权重满足

$$\sum_{k=1}^n w_k = 1$$

可以通过该式验证：

$$\sum_{k=1}^n w_k = \sum_{k=1}^n \frac{R_k^2}{\sum_{i=1}^n R_i^2} = \frac{1}{\sum_{i=1}^n R_i^2} \cdot \sum_{k=1}^n R_k^2 = 1。$$

经过权重的构建，我们可以加权计算综合湍流强度，公式为

$$\bar{I} = \sum_{k=1}^n w_k \cdot I_k = \sum_{k=1}^n \left(\frac{R_k^2}{\sum_{i=1}^n R_i^2} \right) \cdot I_k$$

该公式的物理意义：综合考虑各湍流强度的大小与模型对其的拟合性能，**性能越好，即 R_k^2 越大的湍流强度，对综合结果 I 的贡献越大**，同时通过权重归一化避免某一湍流强度过度主导，实现性能与权重平衡的统一。

其中：

· I_k 为第 k 个湍流强度样本，即湍流强度分量，不同区域、不同情况下的湍流强度， $k=1,2,\dots,n$ ，且 n 为样本总数。

· R_k^2 为第 k 个湍流强度 I_k 对应的决定系数，用与衡量模型对 I_k 的拟合优度或预测性能，且范围 $R^2 \in [0,1)$ 。

· w_k ：第 k 个湍流强度 I_k 的最终权重，最终目标是为了通过 R_k^2 归一化得到，使得满足 $\sum_{k=1}^n w_k = 1$ 。

（2）基于蚁群算法加权湍流动能场规划最优航路

基于**三维蚁群算法以规避高湍流区域**为核心目标，同时考虑部分大小飞行器难以实现蚁群算法中能够像各个方向灵活运动的约束，给蚁群添加严格向前移动条件，通过路径搜索筛选、信息素更新以及最优迭代在指定三维网格空间内寻找从起点到终点的最优路径。本文先规定起点网格坐标(0,0,0)；终点网格坐标(9,9,4)。

三维蚁群算法核心公式：

$$P_{ij}^k(t) = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in allowed_k} [\tau_{il}(t)]^\alpha \cdot [\eta_{il}]^\beta}$$

其中， $P_{ij}^k(t)$ 为蚂蚁 k 从节点 i 到节点 j 的概率； $\tau_{ij}(t)$ 为路径 ij 上的信息素浓度； η_{il} 为启发式信息； α 、 β 为表示调节信息素和启发信息重要程度，本文给定 $\alpha=1$ ， $\beta=3$ ；加以严格向前移动条件

$$\begin{cases} x_{\text{new}} > x_{\text{prev}} \\ y_{\text{new}} > y_{\text{prev}} \\ z_{\text{new}} > z_{\text{prev}} \end{cases}$$

本文运用这个算法每次迭代后，我们仅保留有效路径，迭代结束后，输出最终的全局最优路径，得到最优航路运行图。

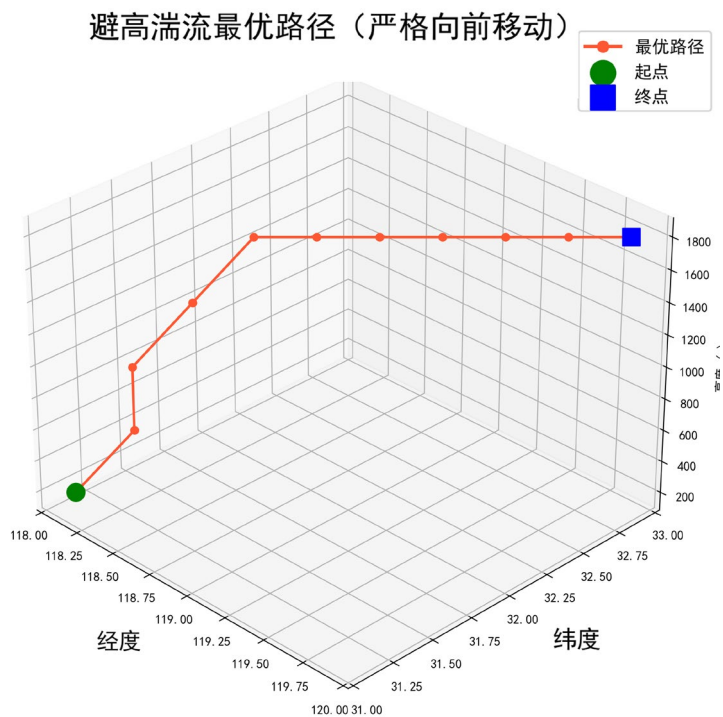


图 35 避开高湍流最优路径图

最优航路路径点如表 5 所示。

表 5 模型 e 最优路径

经度	纬度	高度	TKE
118.1	31.1	200	43.77377
118.3	31.3	600	43.77377
118.3	31.3	1000	14.02512
118.5	31.5	1400	26.71927
118.7	31.7	1800	28.19542
118.9	31.9	1800	24.49227
119.1	32.1	1800	28.55264
119.3	32.3	1800	37.05474
119.5	32.5	1800	26.86942
119.7	32.7	1800	22.0195
119.9	32.9	1800	10.22719

参考文献

- [1] 胡启立,何智浩,张帅,等.吸附动力学模型拟合优度的统计评估及模型比较研究[J].水处理技术,2022, (05):25-29.
- [2] 雷白鸽.基于最优加权平均模型的高程异常拟合分析[J].城市勘测,2025,51(03):118-119.
- [3] 张华,李紫麟,郑文慧,等.基于径向基函数环境载荷传递的计算方法[J].舰船科学技术,2025,47(15):19-23.
- [4] 夏皓,刘杨,贡宇,等.基于空间插值的温室三维温度场构建方法比较研究[J].中国农机化学报,2025,46(06):176-182.
- [5] 胡清海,潘国峰,袁中原.基于蚁群算法的隧道网络风机群优化[J].制冷与空调(四川),2025,39(04):537-540.
- [6] 孙宁,胡云雷,陈宇飞.基于改进 A*算法的轮式巡检机器人路径规划方法[J].湖南大学学报(自然科学版),2025,52(08):103-110.
- [7] 李特,黄孜滢.机器学习在风力发电功率预测中应用综述[J].山东电力技术,2025,52(08):45-55.
- [8] 鞠杨.多机器学习模型下南京市 PM_{2.5} 预测分析[J].环境科学导刊,2025,44(02):46-52.
- [9] 吕泳霖,刘新波,王鹏翔,等.基于集合卡尔曼滤波与长短期记忆神经网络的联合校正方法及其在径流模拟中的应用[J].武汉大学学报(工学版),1-15.
- [10] 王文鹏,何站鹏,巫义锐,等.基于集合卡尔曼滤波的水文模型状态变量反馈校正方法[J].水力发电学报,2024,43(10):17-31.
- [11] 于莹洁.多普勒气象雷达湍流目标检测与信号处理研究[D].西北工业大学,2018.DOI:10.27406/d.cnki.gxbgu.2018.000096.
- [12] 张志标,姜明波,杜智涛,等.风廓线雷达湍流探测应用研究[J].气象水文海洋仪器,2021,38(04):5-7+11.DOI:10.19441/j.cnki.issn1006-009x.2021.04.002.
- [13] Si, Lee, et al. "High-Tc superconductor candidates proposed by machine learning" Machine Learning: Science and Technology 2025.
- [14] Aoi, Watanabe, et al. "douka:A universal platform of data assimilation for materials modeling" The Journal of chemical physics 2025.
- [15] Fang, Q., K. Chu, B. Zhou, X. Chen, Z. Peng, C. Zhang, M. Luo, and C. Zhao, 2023: Observed Turbulent Dissipation Rate in a Landfalling Tropical Cyclone Boundary Layer. J. Atmos. Sci., 80, 1739–1754, <https://doi.org/10.1175/JAS-D-22-0265.1>.
- [16] Sharman, R., C. Tebaldi, G. Wiener, and J. Wolff, 2006: An Integrated Approach to Mid- and Upper-Level Turbulence Forecasting. Wea. Forecasting, 21, 268–287, DOI : <https://doi.org/10.1175/WAF924.1>
- [17] Atsushi Kudo, DEVELOPMENT OF JMA'S NEW TURBULENCE INDEX, Numerical Prediction Division, Japan Meteorological Agency;
- [18] Frehlich, R., and R. Sharman, 2004: Estimates of Turbulence from Numerical Weather Prediction Model Output with Applications to Turbulence Diagnosis and Data Assimilation. Mon. Wea. Rev., 132, 2308–2324, [https://doi.org/10.1175/1520-0493\(2004\)132<2308:EOTFNW>2.0.CO;2](https://doi.org/10.1175/1520-0493(2004)132<2308:EOTFNW>2.0.CO;2).
- [19] 豆包 字节跳动.(2025). (Version X.X) [Computer software]. <https://doubao.com/>.

注：本参赛对部分使用了 AI 工具，主要用于相关雷达设备的认识了解和部分地理信息的收集，作品的核心建模与分析均为独立完成。

附录

```
模型 b 代码
import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedShuffleSplit
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error,
mean_absolute_percentage_error
from pathlib import Path
import itertools
import joblib
import warnings
import xgboost as xgb

class Config:
    INPUT_CSV_PATH = "C:/Users/Ruii/Desktop/py/过程数据/湍流强度/B 模型确认/B 模型合并后的
TKE 数据.csv"
    INPUT_EXCEL_PATH = "C:/Users/Ruii/Desktop/py/过程数据/湍流强度/A 模型确认/A 模型合并后
_TKE 数据.xlsx"
    EXCEL_SHEET_NAME = "Sheet1"
    OUTPUT_DIR = "C:/Users/Ruii/Desktop/py/过程数据/湍流强度/XGBoost 回归/results"
    TIME_FEATURES = ['hour', 'minute']
    CATEGORICAL_FEATURES = []
    NUMERICAL_FEATURES = ['height']
    TEST_SIZE = 0.3
    RANDOM_STATE = 42
    N_ESTIMATORS = 300
    N_JOBS = -1
    LEARNING_RATE = 0.05
    MAX_DEPTH = 8
    SUBSAMPLE = 0.9
    COL_SAMPLE_BYTREE = 0.9
    PLOT_DPI = 300
    PLOT_FORMAT = 'png'
    FIG_SIZE = (10, 8)
    METRICS = ['mse', 'rmse', 'mae', 'r2', 'mape']

class DataLoader:
    def __init__(self, config):
        self.config = config
        self.df_b = None
        self.df_a = None
        self.df_merged = None
        self.feature_names = None
```

```
def load_data(self):
    self.df_b = pd.read_csv(self.config.INPUT_CSV_PATH)
    self.df_a = pd.read_excel(
        self.config.INPUT_EXCEL_PATH,
        sheet_name=self.config.EXCEL_SHEET_NAME
    )
def preprocess_data(self):
    self.df_a.rename(columns={"station_id_label": "station_id"}, inplace=True)
    self.df_b.rename(columns={"station_id_label": "station_id"}, inplace=True)
    self.df_a['obs_time'] = pd.to_datetime(self.df_a['obs_time'])
    self.df_b['obs_time'] = pd.to_datetime(self.df_b['obs_time'])
    for df in [self.df_a, self.df_b]:
        df['hour'] = df['obs_time'].dt.hour
        df['minute'] = df['obs_time'].dt.minute
    for df in [self.df_a, self.df_b]:
        df['station_code'] = (df['station_id'] == 'b').astype(int)
    self.df_merged = pd.merge(
        self.df_b,
        self.df_a[['obs_time', 'height', 'station_id', 'station_code', 'TKE']],
        on=['obs_time', 'height', 'station_id'],
        suffixes=('_b', '_true')
    ).dropna()
def prepare_features(self, df):
    if 'station_code' not in df.columns:
        df['station_code'] = (df['station_id'] == 'b').astype(int)
    all_features = self.config.TIME_FEATURES + self.config.NUMERICAL_FEATURES +
['station_code']
    available_features = [f for f in all_features if f in df.columns]
    self.feature_names = available_features
    return df[available_features].copy()
def get_processed_data(self):
    if self.df_merged is None:
        self.load_data()
        self.preprocess_data()
    return self.df_merged
def get_feature_names(self):
    if self.feature_names is None:
        self.feature_names = self.config.TIME_FEATURES + self.config.NUMERICAL_FEATURES
+ ['station_code']
    return self.feature_names
class ModelTrainer:
    def __init__(self, config):
        self.config = config
        self.model = None
```

```
self.metrics = {}
self.feature_names = None
def set_feature_names(self, feature_names):
    self.feature_names = feature_names
def stratified_train_test_split(self, df):
    sss = StratifiedShuffleSplit(
        n_splits=1,
        test_size=self.config.TEST_SIZE,
        random_state=self.config.RANDOM_STATE
    )
    for train_idx, test_idx in sss.split(df, df['station_id']):
        train_set = df.iloc[train_idx]
        test_set = df.iloc[test_idx]
    return train_set, test_set
def train_model(self, X_train, y_train):
    self.model = xgb.XGBRegressor(
        n_estimators=self.config.N_ESTIMATORS,
        learning_rate=self.config.LEARNING_RATE,
        max_depth=self.config.MAX_DEPTH,
        subsample=self.config.SUBSAMPLE,
        colsample_bytree=self.config.COL_SAMPLE_BYTREE,
        random_state=self.config.RANDOM_STATE,
        n_jobs=self.config.N_JOBS
    )
    self.model.fit(X_train, y_train)
def predict(self, X):
    if self.model is None:
        raise ValueError("模型尚未训练")
    if isinstance(X, np.ndarray) and self.feature_names is not None and len(self.feature_names) ==
X.shape[1]:
        X = pd.DataFrame(X, columns=self.feature_names)
    return self.model.predict(X)
def evaluate_model(self, y_true, y_pred):
    metrics = {}
    if 'mse' in self.config.METRICS:
        metrics['mse'] = mean_squared_error(y_true, y_pred)
    if 'rmse' in self.config.METRICS:
        metrics['rmse'] = np.sqrt(metrics.get('mse', mean_squared_error(y_true, y_pred)))
    if 'mae' in self.config.METRICS:
        metrics['mae'] = mean_absolute_error(y_true, y_pred)
    if 'r2' in self.config.METRICS:
        metrics['r2'] = r2_score(y_true, y_pred)
    if 'mape' in self.config.METRICS:
        try:
```

```
        metrics['mape'] = mean_absolute_percentage_error(y_true, y_pred)
    except:
        metrics['mape'] = np.nan
    self.metrics = metrics
    return metrics
def save_model(self, filepath):
    if self.model:
        joblib.dump(self.model, filepath)
def predict_new_data(self, hour, minute, height, station_code):
    if self.model is None:
        raise ValueError("模型尚未训练")
    if self.feature_names is None:
        raise ValueError("特征名称未设置")
    features_df = pd.DataFrame({
        'hour': [hour],
        'minute': [minute],
        'height': [height],
        'station_code': [station_code]
    })[self.feature_names]
    prediction = self.model.predict(features_df)
    return prediction[0]
class Visualizer:
    def __init__(self, config):
        self.config = config
        self.output_dir = Path(config.OUTPUT_DIR)
        self.output_dir.mkdir(exist_ok=True, parents=True)
def run_single_experiment(config=None):
    if config is None:
        config = Config()
    warnings.filterwarnings("ignore", category=UserWarning,
                           message="X does not have valid feature names")
    data_loader = DataLoader(config)
    model_trainer = ModelTrainer(config)
    visualizer = Visualizer(config)
    df_merged = data_loader.get_processed_data()
    train_set, test_set = model_trainer.stratified_train_test_split(df_merged)
    X_train = data_loader.prepare_features(train_set)
    X_test = data_loader.prepare_features(test_set)
    feature_names = data_loader.get_feature_names()
    model_trainer.set_feature_names(feature_names)
    y_train_b = train_set['TKE_b']
    y_test_true = test_set['TKE_true']
    model_trainer.train_model(X_train, y_train_b)
    y_pred_b = model_trainer.predict(X_test)
```

```
metrics = model_trainer.evaluate_model(y_test_true, y_pred_b)
residuals = y_test_true.values - y_pred_b
model_path = visualizer.output_dir / "xgboost_model.joblib"
model_trainer.save_model(model_path)
results_df = pd.DataFrame({
    'obs_time': test_set['obs_time'].values,
    'height': test_set['height'].values,
    'station_id': test_set['station_id'].values,
    'station_code': test_set['station_code'].values,
    'true_tke': y_test_true.values,
    'predicted_tke': y_pred_b,
    'residuals': residuals
})
results_path = visualizer.output_dir / "prediction_results.csv"
results_df.to_csv(results_path, index=False)
metrics_df = pd.DataFrame([metrics])
metrics_path = visualizer.output_dir / "evaluation_metrics.csv"
metrics_df.to_csv(metrics_path, index=False)
feature_info = pd.DataFrame({
    'feature_name': feature_names,
    'importance': model_trainer.model.feature_importances_
}).sort_values('importance', ascending=False)
feature_path = visualizer.output_dir / "feature_importance.csv"
feature_info.to_csv(feature_path, index=False)
return metrics, results_df
if __name__ == "__main__":
    run_single_experiment()
```

模型 d 代码

```
import os
import re
import glob
import numpy as np
from netCDF4 import Dataset
from datetime import datetime
from tqdm import tqdm
from scipy.interpolate import RegularGridInterpolator
data_dir = r"D:\nc_file"
file_pattern = os.path.join(data_dir, "wrfvar_output_1km.*.nanjing.nc")
lat_min, lat_max = 31, 33
lon_min, lon_max = 118, 120
z_max = 2000
dz = 50
dx = 100
```

```
dy = 100
time_start = datetime(2025, 7, 31, 0, 0)
time_end = datetime(2025, 7, 31, 5, 0)
def parse_time_from_filename(filename):
    match = re.search(r"\.(\d{12})", filename)
    if match:
        timestr = match.group(1)
        return datetime.strptime(timestr, "%Y%m%d%H%M")
    return None
files = sorted(glob.glob(file_pattern))
selected_files = []
for f in files:
    t = parse_time_from_filename(os.path.basename(f))
    if t and time_start <= t <= time_end:
        selected_files.append((f, t))
if not selected_files:
    raise FileNotFoundError("未找到文件")
sample_file = selected_files[0][0]
with Dataset(sample_file) as ds:
    lat = np.squeeze(ds.variables["XLAT"][:])
    lon = np.squeeze(ds.variables["XLONG"][:])
    z = np.squeeze(ds.variables["Z"][0, :, :, :])
lat_mask = (lat >= lat_min) & (lat <= lat_max)
lon_mask = (lon >= lon_min) & (lon <= lon_max)
lat_idx = np.where(lat_mask.any(axis=1))[0]
lon_idx = np.where(lon_mask.any(axis=0))[0]
lat_sel = lat[lat_idx.min():lat_idx.max()+1, lon_idx.min():lon_idx.max()+1]
lon_sel = lon[lat_idx.min():lat_idx.max()+1, lon_idx.min():lon_idx.max()+1]
z_sel = z[:, lat_idx.min():lat_idx.max()+1, lon_idx.min():lon_idx.max()+1]
new_lat = np.linspace(lat_sel.min(), lat_sel.max(), int((lat_sel.max()-lat_sel.min())*111000/dx))
new_lon = np.linspace(lon_sel.min(), lon_sel.max(),
int((lon_sel.max()-lon_sel.min())*111000*np.cos(np.radians(lat_sel.mean()))/dy))
new_z = np.arange(0, z_max+dz, dz)
def interpolate_to_latlonz(data3d, z_coord, new_lat, new_lon, new_z):
    nz, ny, nx = data3d.shape
    if z_coord.ndim == 3:
        z_mean = z_coord.mean(axis=(1,2))
    else:
        z_mean = z_coord
    yi = np.linspace(0, ny-1, ny)
    xi = np.linspace(0, nx-1, nx)
    interp_func = RegularGridInterpolator(
        points=(z_mean, yi, xi),
        values=data3d,
```

```

        bounds_error=False,
        fill_value=np.nan
    )
    grid_z, grid_y, grid_x = np.meshgrid(
        new_z,
        np.linspace(0, ny-1, len(new_lat)),
        np.linspace(0, nx-1, len(new_lon)),
        indexing="ij"
    )
    points = np.vstack([grid_z.ravel(), grid_y.ravel(), grid_x.ravel()]).T
    data_interp = interp_func(points).reshape(len(new_z), len(new_lat), len(new_lon))
    return data_interp.astype(np.float32)
Ua_all, Va_all, Wa_all, TKE_all, times = [], [], [], [], []
for f, t in tqdm(selected_files):
    with Dataset(f) as ds:
        Ua = ds.variables["Ua"][0, :, lat_idx.min():lat_idx.max()+1, lon_idx.min():lon_idx.max()+1]
        Va = ds.variables["Va"][0, :, lat_idx.min():lat_idx.max()+1, lon_idx.min():lon_idx.max()+1]
        Wa = ds.variables["Wa"][0, :, lat_idx.min():lat_idx.max()+1, lon_idx.min():lon_idx.max()+1]
    Ua_i = interpolate_to_latlonz(Ua, z_sel, new_lat, new_lon, new_z)
    Va_i = interpolate_to_latlonz(Va, z_sel, new_lat, new_lon, new_z)
    Wa_i = interpolate_to_latlonz(Wa, z_sel, new_lat, new_lon, new_z)
    Ua_all.append(Ua_i)
    Va_all.append(Va_i)
    Wa_all.append(Wa_i)
    times.append(t)
Ua_all = np.stack(Ua_all)
Va_all = np.stack(Va_all)
Wa_all = np.stack(Wa_all)
Ua_mean = Ua_all.mean(axis=0)
Va_mean = Va_all.mean(axis=0)
Wa_mean = Wa_all.mean(axis=0)
for i in tqdm(range(Ua_all.shape[0])):
    u_prime = Ua_all[i] - Ua_mean
    v_prime = Va_all[i] - Va_mean
    w_prime = Wa_all[i] - Wa_mean
    TKE = 0.5 * (u_prime**2 + v_prime**2 + w_prime**2)
    TKE_all.append(TKE.astype(np.float32))
TKE_all = np.stack(TKE_all)
out_file = os.path.join(data_dir, "wind_interp_tke_00_05.nc")
with Dataset(out_file, "w", format="NETCDF4") as nc_out:
    nc_out.createDimension("time", len(times))
    nc_out.createDimension("z", len(new_z))
    nc_out.createDimension("lat", len(new_lat))
    nc_out.createDimension("lon", len(new_lon))

```

```
time_var = nc_out.createVariable("time", "f8", ("time",))
z_var = nc_out.createVariable("z", "f4", ("z",))
lat_var = nc_out.createVariable("lat", "f4", ("lat",))
lon_var = nc_out.createVariable("lon", "f4", ("lon",))
time_var.units = f"hours since {times[0].strftime('%Y-%m-%d %H:%M:%S')}"
time_var.calendar = "gregorian"
time_var[:] = [(t - times[0]).total_seconds()/3600.0 for t in times]
z_var.units = "m"; z_var[:] = new_z
lat_var.units = "degree_north"; lat_var[:] = new_lat
lon_var.units = "degree_east"; lon_var[:] = new_lon
Ua_var = nc_out.createVariable("Ua", "f4", ("time", "z", "lat", "lon"), zlib=True)
Va_var = nc_out.createVariable("Va", "f4", ("time", "z", "lat", "lon"), zlib=True)
Wa_var = nc_out.createVariable("Wa", "f4", ("time", "z", "lat", "lon"), zlib=True)
TKE_var = nc_out.createVariable("TKE", "f4", ("time", "z", "lat", "lon"), zlib=True)
Ua_var.units="m/s"; Va_var.units="m/s"; Wa_var.units="m/s"; TKE_var.units="m2/s2"
Ua_var[:] = Ua_all; Va_var[:] = Va_all; Wa_var[:] = Wa_all; TKE_var[:] = TKE_all
```

模型 e 代码

```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import warnings
import os
import gc
warnings.filterwarnings('ignore')
class Config:
    def __init__(self):
        self.input_file = 'C:/Users/Ruii/Desktop/py/过程数据 3/S 波段雷达三维数据_sparse_rbf.csv'
        self.output_dir = 'C:/Users/Ruii/Desktop/py/过程数据 3/S 波段 output2.0'
        self.model_save_path = 'C:/Users/Ruii/Desktop/py/过程数据 3/S 波段 output2.0/best_tke_model.pth'
        self.time_steps = 3
        self.batch_size = 64
        self.epochs = 100
        self.learning_rate = 0.001
        self.hidden_dim = 64
        self.num_layers = 2
        self.dropout = 0.2
        self.test_size = 0.2
        self.val_size = 0.2
        self.random_state = 42
        self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
os.makedirs(self.output_dir, exist_ok=True)
self.model_save_path = os.path.join(self.output_dir, 'best_tke_model.pth')
class TKEDataProcessor:
    def __init__(self, config):
        self.config = config
        self.time_steps = config.time_steps
        self.scaler = StandardScaler()
        self.feature_cols = None
        self.target_col = 'TKE'
    def load_data_chunks(self, file_path, chunksize=100000):
        chunks = []
        total_rows = 0
        if file_path.endswith('.csv'):
            with pd.read_csv(file_path, chunksize=chunksize) as reader:
                for chunk in reader:
                    total_rows += len(chunk)
        if file_path.endswith('.csv'):
            reader = pd.read_csv(file_path, chunksize=chunksize)
        else:
            raise ValueError("不支持的文件格式")
        processed_chunks = []
        for chunk in reader:
            required_cols = ['longitude', 'latitude', 'altitude', 'time', 'TKE']
            chunk = chunk[[col for col in required_cols if col in chunk.columns]]
            chunk['time'] = pd.to_datetime(chunk['time'])
            chunk['hour'] = chunk['time'].dt.hour
            chunk['minute'] = chunk['time'].dt.minute
            chunk['second'] = chunk['time'].dt.second
            processed_chunks.append(chunk)
        df = pd.concat(processed_chunks, ignore_index=True)
        return df
    def load_and_preprocess(self, file_path):
        file_size = os.path.getsize(file_path) / (1024 * 1024)
        if file_size > 100:
            df = self.load_data_chunks(file_path)
        else:
            if file_path.endswith('.csv'):
                df = pd.read_csv(file_path)
            elif file_path.endswith('.xlsx') or file_path.endswith('.xls'):
                df = pd.read_excel(file_path)
            else:
                raise ValueError("不支持的文件格式，请使用 CSV 或 Excel 文件")
            required_cols = ['longitude', 'latitude', 'altitude', 'time', 'TKE']
            df = df[[col for col in required_cols if col in df.columns]]
```

```
df['time'] = pd.to_datetime(df['time'])
df['hour'] = df['time'].dt.hour
df['minute'] = df['time'].dt.minute
df['second'] = df['time'].dt.second
df['longitude'] = df['longitude'].round(2)
df['latitude'] = df['latitude'].round(2)
df['altitude'] = (df['altitude'] // 50) * 50
return df
def analyze_data_structure(self, df):
    df_sorted = df.sort_values('time')
    time_diffs = df_sorted['time'].diff().dropna()
    unique_diffs = time_diffs.unique()
    location_stats = df.groupby(['longitude', 'latitude', 'altitude']).size()
def create_sequences_original_time(self, df):
    groups = list(df.groupby(['longitude', 'latitude', 'altitude']))
    group_lengths = []
    for (lon, lat, alt), group in groups:
        group_lengths.append(len(group))
    X_sequences, y_targets, spatial_info, time_info = [], [], [], []
    valid_sequences = 0
    for (lon, lat, alt), group in groups:
        group_sorted = group.sort_values('time').reset_index(drop=True)
        current_group_length = len(group_sorted)
        if current_group_length < self.time_steps + 1:
            continue
        time_diffs = group_sorted['time'].diff().dropna()
        avg_diff = time_diffs.mean()
        max_diff = time_diffs.max()
        if max_diff > avg_diff * 5:
            pass
    for (lon, lat, alt), group in groups:
        group = group.sort_values('time')
        if len(group) < self.time_steps + 1:
            continue
        tke_values = group['TKE'].values
        hours = group['hour'].values
        minutes = group['minute'].values
        seconds = group['second'].values
        times = group['time'].values
        n_sequences = len(group) - self.time_steps
        if n_sequences <= 0:
            continue
        for i in range(n_sequences):
            sequence_features = []
```

```
        for j in range(self.time_steps):
            features = [lon, lat, alt, hours[i + j], minutes[i + j], seconds[i + j], tke_values[i + j]]
            sequence_features.append(features)
        X_sequences.append(sequence_features)
        y_targets.append(tke_values[i + self.time_steps])
        spatial_info.append((lon, lat, alt))
        time_info.append(times[i + self.time_steps])
        valid_sequences += 1

X_array = np.array(X_sequences, dtype=np.float32)
y_array = np.array(y_targets, dtype=np.float32)
del X_sequences, y_targets
gc.collect()
return X_array, y_array, spatial_info, time_info

def prepare_data(self, file_path):
    df = self.load_and_preprocess(file_path)
    if df is None or len(df) == 0:
        return np.array([]), np.array([]), [], [], []
    self.feature_cols = ['longitude', 'latitude', 'altitude', 'hour', 'minute', 'second', 'TKE']
    self.analyze_data_structure(df)
    X, y, spatial_info, time_info = self.create_sequences_original_time(df)
    if X.shape[0] == 0:
        return np.array([]), np.array([]), [], [], []
    X_resaped = X.reshape(-1, len(self.feature_cols))
    X_scaled = self.scaler.fit_transform(X_resaped)
    X_scaled = X_scaled.reshape(X.shape).astype(np.float32)
    return X_scaled, y, spatial_info, time_info, self.feature_cols

class TKEDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.from_numpy(X).float()
        self.y = torch.from_numpy(y).float().view(-1, 1)
    def __len__(self):
        return len(self.X)
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

class TKE_LSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim=64, output_dim=1, num_layers=2, dropout=0.2):
        super(TKE_LSTM, self).__init__()
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers
        self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers,
                             batch_first=True, dropout=dropout if num_layers > 1 else 0)
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim, output_dim)
        self.batch_norm = nn.BatchNorm1d(hidden_dim)
```

```
def forward(self, x):
    lstm_out, (hn, cn) = self.lstm(x)
    last_output = lstm_out[:, -1, :]
    last_output = self.batch_norm(last_output)
    last_output = self.dropout(last_output)
    output = self.fc(last_output)
    return output

class TKETrainer:
    def __init__(self, model, device, config):
        self.model = model.to(device)
        self.device = device
        self.config = config
        self.criterion = nn.MSELoss()
        self.train_losses = []
        self.val_losses = []

    def train_epoch(self, dataloader, optimizer):
        self.model.train()
        total_loss = 0
        for batch_X, batch_y in dataloader:
            batch_X, batch_y = batch_X.to(self.device), batch_y.to(self.device)
            optimizer.zero_grad()
            outputs = self.model(batch_X)
            loss = self.criterion(outputs, batch_y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(self.model.parameters(), max_norm=1.0)
            optimizer.step()
            total_loss += loss.item()
        return total_loss / len(dataloader)

    def validate(self, dataloader):
        self.model.eval()
        total_loss = 0
        with torch.no_grad():
            for batch_X, batch_y in dataloader:
                batch_X, batch_y = batch_X.to(self.device), batch_y.to(self.device)
                outputs = self.model(batch_X)
                loss = self.criterion(outputs, batch_y)
                total_loss += loss.item()
        return total_loss / len(dataloader)

    def train(self, train_loader, val_loader, optimizer, epochs=100, patience=10):
        best_val_loss = float('inf')
        patience_counter = 0
        for epoch in range(epochs):
            train_loss = self.train_epoch(train_loader, optimizer)
            val_loss = self.validate(val_loader)
```

```
        self.train_losses.append(train_loss)
        self.val_losses.append(val_loss)
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            patience_counter = 0
            torch.save(self.model.state_dict(), self.config.model_save_path)
        else:
            patience_counter += 1
        if patience_counter >= patience:
            break

class TKEEvaluator:
    def __init__(self, model, device, config):
        self.model = model.to(device)
        self.device = device
        self.config = config
    def evaluate(self, dataloader):
        self.model.eval()
        predictions, targets = [], []
        with torch.no_grad():
            for batch_X, batch_y in dataloader:
                batch_X, batch_y = batch_X.to(self.device), batch_y.to(self.device)
                outputs = self.model(batch_X)
                predictions.extend(outputs.cpu().numpy())
                targets.extend(batch_y.cpu().numpy())
        predictions = np.array(predictions).flatten()
        targets = np.array(targets).flatten()
        return predictions, targets
    def calculate_metrics(self, predictions, targets):
        from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
        mse = mean_squared_error(targets, predictions)
        rmse = np.sqrt(mse)
        mae = mean_absolute_error(targets, predictions)
        r2 = r2_score(targets, predictions)
        return {
            'MSE': mse,
            'RMSE': rmse,
            'MAE': mae,
            'R2': r2
        }
    def save_results(self, predictions, targets, spatial_info, time_info, save_path):
        results_df = pd.DataFrame({
            'longitude': [info[0] for info in spatial_info],
            'latitude': [info[1] for info in spatial_info],
```

```
        'altitude': [info[2] for info in spatial_info],
        'prediction_time': time_info,
        'true_tke': targets,
        'predicted_tke': predictions,
        'residual': targets - predictions
    })
    results_df.to_csv(save_path, index=False)
    return results_df

def main():
    config = Config()
    torch.manual_seed(config.random_state)
    np.random.seed(config.random_state)
    try:
        processor = TKEDDataProcessor(config)
        X, y, spatial_info, time_info, feature_cols = processor.prepare_data(config.input_file)
        if X.shape[0] == 0:
            return
        X_train, X_test, y_train, y_test, spatial_train, spatial_test, time_train, time_test = train_test_split(
            X, y, spatial_info, time_info, test_size=config.test_size, random_state=config.random_state
        )
        X_train, X_val, y_train, y_val, spatial_train, spatial_val, time_train, time_val = train_test_split(
            X_train, y_train, spatial_train, time_train, test_size=config.val_size,
            random_state=config.random_state
        )
        del X, y, spatial_info, time_info
        gc.collect()
        train_dataset = TKEDataset(X_train, y_train)
        val_dataset = TKEDataset(X_val, y_val)
        test_dataset = TKEDataset(X_test, y_test)
        num_workers = min(4, os.cpu_count() // 2)
        train_loader = DataLoader(train_dataset, batch_size=config.batch_size, shuffle=True,
                                   num_workers=num_workers, pin_memory=True)
        val_loader = DataLoader(val_dataset, batch_size=config.batch_size, shuffle=False,
                                   num_workers=num_workers, pin_memory=True)
        test_loader = DataLoader(test_dataset, batch_size=config.batch_size, shuffle=False,
                                   num_workers=num_workers, pin_memory=True)

        input_dim = len(feature_cols)
        model = TKE_LSTM(
            input_dim=input_dim,
            hidden_dim=config.hidden_dim,
            output_dim=1,
            num_layers=config.num_layers,
            dropout=config.dropout
        )
```

```
trainer = TKETrainer(model, config.device, config)
optimizer = torch.optim.Adam(model.parameters(), lr=config.learning_rate, weight_decay=1e-5)
trainer.train(train_loader, val_loader, optimizer, epochs=config.epochs, patience=15)
model.load_state_dict(torch.load(config.model_save_path))
evaluator = TKEEvaluator(model, config.device, config)
test_predictions, test_targets = evaluator.evaluate(test_loader)
metrics = evaluator.calculate_metrics(test_predictions, test_targets)
results_path = os.path.join(config.output_dir, 'original_time_prediction_results.csv')
evaluator.save_results(test_predictions, test_targets, spatial_test, time_test, results_path)
metrics_df = pd.DataFrame([metrics])
metrics_path = os.path.join(config.output_dir, 'model_metrics.csv')
metrics_df.to_csv(metrics_path, index=False)
except Exception as e:
    import traceback
    traceback.print_exc()
if __name__ == "__main__":
    main()
```

模型 e 代码

```
import numpy as np
import pandas as pd
from scipy.spatial import KDTree
import os
INPUT_CSV_PATH = "C:/Users/Ruii/Desktop/py/过程数据 3/E 模型的 TKE 预测结果.csv"
OUTPUT_DIR = "C:/Users/Ruii/Desktop/py/过程数据 3/E 模型最优路径"
PLOT_FILENAME = "蚁群_tke_path_3d_forward.png"

LON_MIN, LON_MAX = 118, 120
LAT_MIN, LAT_MAX = 31, 33
ALT_MIN, ALT_MAX = 0, 2000
GRID_SIZE = (10, 10, 5)

START_POINT = (0, 0, 0)
END_POINT = (9, 9, 4)
N_ANTS = 30
MAX_ITER = 50
ALPHA = 1
BETA = 3
RHO = 0.5
Q = 100

PLOT_DPI = 300
DISPLAY_LAYER = 0
class AntColony3DForward:
```

```
def __init__(self, grid, start, end, n_ants=50, max_iter=100,
              alpha=1, beta=2, rho=0.5, q=100):
    self.grid = grid
    self.start = start
    self.end = end
    self.n_ants = n_ants
    self.max_iter = max_iter
    self.alpha = alpha
    self.beta = beta
    self.rho = rho
    self.q = q
    self.grid_shape = grid.shape
    self.pheromone = np.ones(self.grid_shape) * 0.1
def heuristic(self, pos):
    return 1 / (self.grid[pos] + 1e-5)
def get_valid_forward_neighbors(self, current_pos, visited_path):
    valid_neighbors = []
    x_curr, y_curr, z_curr = current_pos
    if visited_path:
        min_x = min(p[0] for p in visited_path)
        min_y = min(p[1] for p in visited_path)
        min_z = min(p[2] for p in visited_path)
    else:
        min_x, min_y, min_z = x_curr, y_curr, z_curr
    for dx in [0, 1]:
        for dy in [0, 1]:
            for dz in [0, 1]:
                if dx == 0 and dy == 0 and dz == 0:
                    continue
                neighbor_pos = (x_curr + dx, y_curr + dy, z_curr + dz)
                if not (0 <= neighbor_pos[0] < self.grid_shape[0] and
                        0 <= neighbor_pos[1] < self.grid_shape[1] and
                        0 <= neighbor_pos[2] < self.grid_shape[2]):
                    continue
                if neighbor_pos in visited_path:
                    continue
                nx, ny, nz = neighbor_pos
                if (nx > min_x and ny > min_y and nz > min_z):
                    valid_neighbors.append(neighbor_pos)
    return valid_neighbors
def run(self):
    best_path = None
    best_cost = float('inf')
    for iteration in range(self.max_iter):
```

```
all_valid_paths = []
all_valid_costs = []
invalid_count = 0
for ant in range(self.n_ants):
    path = [self.start]
    current_cost = self.grid[self.start]
    current_pos = self.start
    while True:
        if current_pos == self.end:
            all_valid_paths.append(path.copy())
            all_valid_costs.append(current_cost)
            if current_cost < best_cost:
                best_cost = current_cost
                best_path = path.copy()
            break
        valid_neighbors = self.get_valid_forward_neighbors(current_pos, path)
        if not valid_neighbors:
            invalid_count += 1
            break
        neighbor_probs = []
        for neighbor_pos in valid_neighbors:
            pheromone_val = self.pheromone[neighbor_pos]
            heuristic_val = self.heuristic(neighbor_pos)
            transfer_prob = (pheromone_val ** self.alpha) * (heuristic_val ** self.beta)
            neighbor_probs.append(transfer_prob)
        neighbor_probs = np.array(neighbor_probs)
        if neighbor_probs.sum() > 0:
            neighbor_probs /= neighbor_probs.sum()
        else:
            neighbor_probs = np.ones(len(valid_neighbors)) / len(valid_neighbors)
        selected_idx = np.random.choice(len(valid_neighbors), p=neighbor_probs)
        next_pos = valid_neighbors[selected_idx]
        path.append(next_pos)
        current_cost += self.grid[next_pos]
        current_pos = next_pos
    self.pheromone *= (1 - self.rho)
    for path, cost in zip(all_valid_paths, all_valid_costs):
        deposit_amount = self.q / (cost + 1e-5)
        for node_pos in path:
            self.pheromone[node_pos] += deposit_amount
    return best_path, best_cost
def fill_nan_with_nearest(grid):
    non_nan_pos = np.argwhere(~np.isnan(grid))
    non_nan_vals = grid[~np.isnan(grid)]
```

```

if len(non_nan_pos) == 0 or len(non_nan_pos) == np.prod(grid.shape):
    return grid
kdtree = KDTree(non_nan_pos)
nan_pos = np.argwhere(np.isnan(grid))
_, nearest_indices = kdtree.query(nan_pos)
grid[tuple(nan_pos.T)] = non_nan_vals[nearest_indices]
return grid

def grid_to_coords(path, lon_min, lon_max, lat_min, lat_max,
                    alt_min, alt_max, grid_size):
    lon_step = (lon_max - lon_min) / grid_size[0]
    lat_step = (lat_max - lat_min) / grid_size[1]
    alt_step = (alt_max - alt_min) / grid_size[2]
    physical_coords = []
    for (lon_idx, lat_idx, alt_idx) in path:
        lon = lon_min + (lon_idx + 0.5) * lon_step
        lat = lat_min + (lat_idx + 0.5) * lat_step
        alt = alt_min + (alt_idx + 0.5) * alt_step
        physical_coords.append((lon, lat, alt))
    return np.array(physical_coords)

def verify_forward_path(path):
    if not path or len(path) < 2:
        return True
    for i in range(1, len(path)):
        current = path[i]
        for j in range(i):
            prev = path[j]
            if not (current[0] > prev[0] and current[1] > prev[1] and current[2] > prev[2]):
                return False
    return True

def main():
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    df = pd.read_csv(INPUT_CSV_PATH)
    required_cols = ['longitude', 'latitude', 'altitude', 'predicted_tke']
    if not all(col in df.columns for col in required_cols):
        raise ValueError(f"CSV 文件缺少必要列，需包含：{required_cols}")
    lon_step = (LON_MAX - LON_MIN) / GRID_SIZE[0]
    lat_step = (LAT_MAX - LAT_MIN) / GRID_SIZE[1]
    alt_step = (ALT_MAX - ALT_MIN) / GRID_SIZE[2]
    tke_grid = np.full(GRID_SIZE, np.nan)
    count_grid = np.zeros(GRID_SIZE, dtype=int)
    for idx, row in df.iterrows():
        try:
            lon = row['longitude']
            lat = row['latitude']

```

```
        alt = row['altitude']
        tke = row['predicted_tke']
        if np.isnan(tke) or tke < 0:
            continue
        lon_idx = int((lon - LON_MIN) // lon_step)
        lat_idx = int((lat - LAT_MIN) // lat_step)
        alt_idx = int((alt - ALT_MIN) // alt_step)
        lon_idx = min(lon_idx, GRID_SIZE[0] - 1)
        lat_idx = min(lat_idx, GRID_SIZE[1] - 1)
        alt_idx = min(alt_idx, GRID_SIZE[2] - 1)
        if np.isnan(tke_grid[lon_idx, lat_idx, alt_idx]):
            tke_grid[lon_idx, lat_idx, alt_idx] = tke
        else:
            tke_grid[lon_idx, lat_idx, alt_idx] += tke
            count_grid[lon_idx, lat_idx, alt_idx] += 1
    except Exception:
        continue
tke_grid = np.where(count_grid > 0, tke_grid / count_grid, np.nan)
tke_grid = fill_nan_with_nearest(tke_grid)
aco = AntColony3DForward(
    grid=tke_grid,
    start=START_POINT,
    end=END_POINT,
    n_ants=N_ANTS,
    max_iter=MAX_ITER,
    alpha=ALPHA,
    beta=BETA,
    rho=RHO,
    q=Q
)
best_path, best_cost = aco.run()
if best_path:
    is_valid = verify_forward_path(best_path)
    path_coords = grid_to_coords(
        best_path,
        LON_MIN, LON_MAX,
        LAT_MIN, LAT_MAX,
        ALT_MIN, ALT_MAX,
        GRID_SIZE
    )
    path_df = pd.DataFrame({
        'longitude': path_coords[:, 0],
        'latitude': path_coords[:, 1],
        'altitude': path_coords[:, 2],
```

```
        'grid_lon_idx': [p[0] for p in best_path],
        'grid_lat_idx': [p[1] for p in best_path],
        'grid_alt_idx': [p[2] for p in best_path],
        'tke_value': [tke_grid[p] for p in best_path]
    })
    path_csv_path = os.path.join(OUTPUT_DIR, "最优路径详细数据_向前移动.csv")
    path_df.to_csv(path_csv_path, index=False, encoding='utf-8-sig')
else:
    raise Exception("寻优失败：未找到从起点到终点的有效路径")
if __name__ == "__main__":
    main()
```